

=== ПРАВИЛА ОБРАЩЕНИЯ С КОДОМ ===

1. Не переименовывать и не удалять переменные или методы в процессе редактирования.
2. Если выдаваемый тобой код более 3500 символов - разбей его на части.

=== СТРУКТУРА ВЫБРАННЫХ КАТАЛОГОВ ===

```
[Файл] Mikrotik\MikrotikConnfig.cs
[Файл] Mikrotik\MikrotikSyncCore.cs
[Файл] Mikrotik\Networks.cs
[Файл] Properties\Settings.Designer.cs
[Файл] Properties\Settings.settings
[Файл] App.config
[Файл] App.xaml
[Файл] App.xaml.cs
[Файл] AssemblyInfo.cs
[Файл] EliteTheme.xaml
[Файл] frmMikrotikRestConfig.xaml
[Файл] frmMikrotikRestConfig.xaml.cs
[Файл] frmSettings.xaml
[Файл] frmSettings.xaml.cs
[Файл] MainWindow.xaml
[Файл] MainWindow.xaml.cs
[Файл] wiseIPList.cs
```

=== СОДЕРЖИМОЕ ФАЙЛОВ ===

---Mikrotik\MikrotikConnfig.cs---

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Security.Cryptography;
using System.Text;
using System.Text.Json;

namespace EDIPSearch.Models;

public class MikrotikConfig
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Name { get; set; } = "Новый роутер";
    public string InternalIp { get; set; } = "192.168.88.1";
    public int Port { get; set; } = 443;
    public bool UseSsl { get; set; } = true;
    public string Username { get; set; } = "admin";
    public string EncryptedPassword { get; set; } = string.Empty; // Храним только в зашифрованном виде
    public string TargetAddressList { get; set; } = "ED"; // Имя списка на роутере
    public string WanInterface { get; set; } = "ether1";
    public override string ToString()
    {
        return Name;
    }
    // Свойство для работы в UI (не сериализуется в JSON напрямую)
    [System.Text.Json.Serialization.JsonIgnore]
    public string Password
    {
        get => Decrypt(EncryptedPassword);
        set => EncryptedPassword = Encrypt(value);
    }

    #region DPAPI Encryption
    private static readonly byte[] Entropy = { 0x05, 0x56, 0x0B, 0x11, 0x03, 0x13 }; // Дополнительная соль

    private string Encrypt(string plainText)
    {
        if (string.IsNullOrEmpty(plainText)) return string.Empty;
        byte[] plainBytes = Encoding.UTF8.GetBytes(plainText);
        byte[] encryptedBytes = ProtectedData.Protect(plainBytes, Entropy,
            DataProtectionScope.CurrentUser);
        return Convert.ToBase64String(encryptedBytes);
    }

    private string Decrypt(string encryptedText)
    {

```

```

if (string.IsNullOrEmpty(encryptedText)) return string.Empty;
try
{
byte[] encryptedBytes = Convert.FromBase64String(encryptedText);
byte[] plainBytes = ProtectedData.Unprotect(encryptedBytes, Entropy,
DataProtectionScope.CurrentUser);
return Encoding.UTF8.GetString(plainBytes);
}
catch { return "ОШИБКА_ДЕШИФРОВАНИЯ"; }
}
#endregion
}

public static class RouterStorage
{
private static readonly string FolderPath =
Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData), "EDIPSearch");
private static readonly string FilePath = Path.Combine(FolderPath, "routers.dat");

public static void Save(List<MikrotikConfig> routers)
{
if (!Directory.Exists(FolderPath)) Directory.CreateDirectory(FolderPath);
string json = JsonSerializer.Serialize(routers, new JsonSerializerOptions { WriteIndented = true
});

// Шифруем весь файл конфигурации целиком для максимальной безопасности (IP, порты, логины)
byte[] plainBytes = Encoding.UTF8.GetBytes(json);
byte[] encryptedBytes = ProtectedData.Protect(plainBytes, null, DataProtectionScope.CurrentUser);
File.WriteAllBytes(FilePath, encryptedBytes);
}

public static List<MikrotikConfig> Load()
{
if (!File.Exists(FilePath)) return new List<MikrotikConfig>();
try
{
byte[] encryptedBytes = File.ReadAllBytes(FilePath);
byte[] plainBytes = ProtectedData.Unprotect(encryptedBytes, null, DataProtectionScope.CurrentUser);
string json = Encoding.UTF8.GetString(plainBytes);
return JsonSerializer.Deserialize<List<MikrotikConfig>>(json) ?? new List<MikrotikConfig>();
}
catch
{
return new List<MikrotikConfig>(); // Если файл поврежден или чужой пользователь
}
}
}

---Mikrotik\MikrotikSyncCore.cs---
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using EDIPSearch.Models;
using EDIPSearch.Network;
using ipinpool; // Твое пространство имен для wiseIPList и IPclass

namespace EDIPSearch.Core;

public class MikrotikSyncCore
{
// Список для фильтра-исключений (WAN IP-адреса наших роутеров)
public List<IPclass> RouterWanExclusions { get; private set; } = new List<IPclass>();

// Единый сквозной список адресов, построенный на основе данных со всех роутеров
public wiseIPList GlobalRouterCache { get; private set; } = new wiseIPList();

/// <summary>
/// Фаза 1: Опрос роутеров, сбор WAN IP и построение единой базы существующих адресов
/// </summary>
/// <param name="configuredRouters">Список настроек роутеров, загруженный из RouterStorage</param>

```

```

public async Task InitializeAndFetchAsync(List<MikrotikConfig> configuredRouters)
{
    // Сбрасываем старые данные перед новым опросом
    RouterWanExclusions.Clear();
    GlobalRouterCache = new wiseIPList();

    foreach (var config in configuredRouters)
    {
        var client = new MikrotikRestClient(config);

        // 1. Проверяем связь, если роутер недоступен — пропускаем его во избежание зависаний
        var status = await client.TestConnectionAsync();
        if (status != ConnectionStatus.Success)
        {
            // Роутер оффлайн или неверный пароль, пропускаем его обработку
            continue;
        }

        // 2. Опрашиваем роутер на предмет внешнего IP (интерфейс ether1)
        IPclass? wanIp = await client.GetInterfaceIpAsync();
        if (wanIp != null)
        {
            // Явно выставляем маску /32 (PoolSize = 32), чтобы в исключения
            // попал только сам роутер, а не вся подсеть провайдера вокруг него.
            wanIp.PoolSize = 32;
            RouterWanExclusions.Add(wanIp);
        }

        // 3. Запрашиваем содержимое целевого списка (например, "ED") с этого микротика
        List<string> rawAddresses = await client.GetRawAddressesFromListAsync(config.TargetAddressList);

        foreach (var rawAddr in rawAddresses)
        {
            // Используем твой оригинальный метод парсинга строк
            IPclass? ipObj = IPclass.Parse(rawAddr);
            if (ipObj != null)
            {
                // Твой wiseIPList сам внутри себя разберется с дубликатами
                // и укрупнением диапазонов, строя чистую сводную таблицу
                GlobalRouterCache.AddAddress(ipObj);
            }
        }
    }

    // На выходе из метода у нас готов GlobalRouterCache.
    // Теперь мы можем передать его в твою основную логику приложения,
    // чтобы использовать как глобальный фильтр при чтении логов.
}

/// <summary>
/// Фаза 2: Синхронизация обратно на роутеры (отправка того, чего на них не хватает)
/// Вызывается, когда в процессе парсинга логов накопились новые адреса.
/// </summary>
/// <summary>
/// Прямой и лаконичный пуш всего мастер-списка на все роутеры без лишних сверок
/// </summary>
public async Task PushNewAddressesToRoutersAsync(List<MikrotikConfig> configuredRouters, wiseIPList
masterList)
{
    foreach (var config in configuredRouters)
    {
        var client = new MikrotikRestClient(config);

        // Быстрая проверка: если роутер выключен — не виснем, идем к следующему
        var status = await client.TestConnectionAsync().ConfigureAwait(false);
        if (status != ConnectionStatus.Success) continue;

        // Просто бежим по всей твоей готовой таблице IPclass и шлем её в роутер!
        foreach (IPclass ipObj in masterList.ipTable)
        {
            string ipStr = ipObj.ToString();

```

```
// Если это хост /32 (или PoolSize == 0 в твоей логике) — ставим 7 дней, иначе заливаем подсеть
навсегда
string? timeoutParam = (ipObj.PoolSize == 32 || ipObj.PoolSize == 0) ? "7d" : null;

// Отправляем напрямую. Mikrotik сам пропустит этот IP, если он там уже был активен.
await client.AddAddressAsync(ipStr, config.TargetAddressList, timeoutParam).ConfigureAwait(false);
}
}
}

}
```

```
---Mikrotik\Networks.cs---
```

```
using EDIPSearch.Models;
using ipinpool;
using System;
using System.Net;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Text;
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Threading.Tasks;
```

```
namespace EDIPSearch.Network;
```

```
public enum ConnectionStatus { Success, Unauthorized, Unreachable }
```

```
public class MikrotikRestClient
{
    private readonly HttpClient _httpClient;
    public MikrotikConfig Config { get; }
    private class AddressListEntry
    {
        [JsonPropertyName("address")] public string Address { get; set; } = string.Empty;
        [JsonPropertyName("disabled")] public string Disabled { get; set; } = "false";
        [JsonPropertyName("timeout")] public string Timeout { get; set; } = string.Empty; // ДОБАВЛЕНО:
        Сюда прилетит время вроде "5d04:12:30" или "01:20:00"
    }
}
```

```
public MikrotikRestClient(MikrotikConfig config)
{
    Config = config;
    var handler = new HttpClientHandler
    {
        ServerCertificateCustomValidationCallback = (m, c, ch, e) => true
    };
}
```

```
_httpClient = new HttpClient(handler) { Timeout = TimeSpan.FromSeconds(5) };
string protocol = config.UseSsl ? "https" : "http";
_httpClient.BaseAddress = new Uri($"{protocol}://{config.InternalIp}:{config.Port}/rest/");
```

```
var authToken =
    Convert.ToBase64String(Encoding.UTF8.GetBytes($"{config.Username}:{config.Password}"));
_httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Basic",
    authToken);
}
```

```
#region Вспомогательные JSON-модели
```

```
private class UniqueListEntry
{
    [JsonPropertyName("list")] public string ListName { get; set; } = string.Empty;
}
}
```

```
private class InterfaceAddressEntry
{
}
```

```

[JsonPropertyName("address")] public string Address { get; set; } = string.Empty; // Возвращает
"IP/маска"
}

private class InterfaceEntry
{
[JsonPropertyName("name")] public string Name { get; set; } = string.Empty;
}
#endregion

/// <summary>
/// Запрос ВСЕХ интерфейсов роутера (для выпадающего списка выбора WAN)
/// </summary>
public async Task<List<string>> GetInterfaceNamesAsync()
{
var interfaceNames = new List<string>();
try
{
// Запрашиваем только свойство 'name' со всех интерфейсов роутера
var response = await _httpClient.GetAsync("interface?.proplist=name");
if (!response.IsSuccessStatusCode) return interfaceNames;

var json = await response.Content.ReadAsStringAsync();
var entries = JsonSerializer.Deserialize<List<InterfaceEntry>>(json);

if (entries != null)
{
foreach (var entry in entries)
{
if (!string.IsNullOrEmpty(entry.Name))
if (entry.Name.Trim().ToLower() != "lo")
interfaceNames.Add(entry.Name.Trim());
}
}
}
catch { }
return interfaceNames;
}

public async Task<List<string>> GetAddressListNamesAsync()
{
var listNames = new List<string>();
try
{
// Фильтруем вывод через .proplist=list, чтобы забирать только имена списков
var response = await _httpClient.GetAsync("ip/firewall/address-list?.proplist=list");
if (!response.IsSuccessStatusCode) return listNames;

var json = await response.Content.ReadAsStringAsync();
var entries = JsonSerializer.Deserialize<List<UniqueListEntry>>(json);

if (entries != null)
{
var uniqueSet = new HashSet<string>();
foreach (var entry in entries)
{
if (!string.IsNullOrEmpty(entry.ListName))
uniqueSet.Add(entry.ListName);
}
listNames.AddRange(uniqueSet);
}
}
catch { }
return listNames;
}

/// <summary>
/// Безопасное получение точного внешнего IP-адреса роутера хостом /32.
/// Использует явно указанный пользователем интерфейс из конфигурации.
/// </summary>
public async Task<IPclass?> GetInterfaceIpAsync()
{

```

```

string? detectedRawIp = null;
string targetIface = Config.WanInterface; // Берем имя интерфейса из конфига роутера

// Шаг 1: Запрашиваем IP строго с указанного интерфейса
try
{
var response = await _httpClient.GetAsync($"ip/address?interface={targetIface}");
if (response.IsSuccessStatusCode)
{
var json = await response.Content.ReadAsStringAsync();
var entries = JsonSerializer.Deserialize<List<InterfaceAddressEntry>>(json);

if (entries != null && entries.Count > 0 && !string.IsNullOrEmpty(entries[0].Address))
{
detectedRawIp = entries[0].Address.Trim();
}
}
}
catch { /* Ошибка запроса к роутеру */ }

// Шаг 2: Если адрес найден, проверяем на "серость" и отсекаем маску
if (!string.IsNullOrEmpty(detectedRawIp))
{
int slashIndex = detectedRawIp.IndexOf('/');
string cleanIp = slashIndex > 0 ? detectedRawIp.Substring(0, slashIndex) : detectedRawIp;

if (!IsPrivateIp(cleanIp))
{
// Адрес белый, принудительно возвращаем хост /32 для твоего пула
return IPclass.Parse($"{cleanIp}/32");
}
// Если адрес приватный (за NAT провайдера), то локальный адрес интерфейса (например 10.x.x.x)
// нам не поможет предотвратить петлю, так как лог игры зафиксирует глобальный IP. Идем к Шагу 3.
}

// Шаг 3: Интерфейс пуст или за NAT — стучимся на внешний сервис через этот роутер
try
{
using (var externalClient = new HttpClient { Timeout = TimeSpan.FromSeconds(3) })
{
string publicIp = await externalClient.GetStringAsync("https://icanhazip.com");
publicIp = publicIp.Trim();

if (!string.IsNullOrEmpty(publicIp) && System.Net.IPAddress.TryParse(publicIp, out _))
{
return IPclass.Parse($"{publicIp}/32");
}
}
}
catch { }

return null;
}

/// <summary>
/// Проверка, является ли IP-адрес приватным (серым) в рамках стандартов RFC 1918 и CGNAT RFC 6598
/// </summary>
private bool IsPrivateIp(string ipAddress)
{
if (!IPAddress.TryParse(ipAddress, out IPAddress? ip)) return true;
byte[] bytes = ip.GetAddressBytes();

if (bytes.Length != 4) return true; // Нас интересует только IPv4

// 10.0.0.0/8
if (bytes[0] == 10) return true;

// 172.16.0.0/12

```

```

if (bytes[0] == 172 && (bytes[1] >= 16 && bytes[1] <= 31)) return true;

// 192.168.0.0/16
if (bytes[0] == 192 && bytes[1] == 168) return true;

// 100.64.0.0/10 (CGNAT провайдеров — очень частая история на домашних тарифах)
if (bytes[0] == 100 && (bytes[1] >= 64 && bytes[1] <= 127)) return true;

return false;
}

/// <summary>
/// 4. Запрос содержимого конкретного списка (возвращает ТОЛЬКО АКТИВНЫЕ строки-адреса из Mikrotik)
/// </summary>
public async Task<List<string>> GetRawAddressesFromListAsync(string listName)
{
    var rawAddresses = new List<string>();
    try
    {
        // Запрашиваем свойства address и disabled со списка файрвола
        var response = await _httpClient.GetAsync($"ip/firewall/address-
list?list={listName}&.proplist=address,disabled").ConfigureAwait(false);
        if (!response.IsSuccessStatusCode) return rawAddresses;

        var json = await response.Content.ReadAsStringAsync().ConfigureAwait(false);
        using (JsonDocument doc = JsonDocument.Parse(json))
        {
            if (doc.RootElement.ValueKind == JsonValueKind.Array)
            {
                foreach (var element in doc.RootElement.EnumerateArray())
                {
                    // Проверяем, отключена ли запись
                    bool isDisabled = false;
                    if (element.TryGetProperty("disabled", out JsonElement disabledProp))
                    {
                        string? disabledStr = disabledProp.GetString();
                        // Mikrotik возвращает "true" для отключенных записей
                        if (!string.IsNullOrEmpty(disabledStr) && disabledStr.Equals("true",
                            StringComparison.OrdinalIgnoreCase))
                        {
                            isDisabled = true;
                        }
                    }

                    // Если запись активна (не disabled), забираем её IP-адрес
                    if (!isDisabled && element.TryGetProperty("address", out JsonElement addrProp))
                    {
                        string? addr = addrProp.GetString();
                        if (!string.IsNullOrEmpty(addr))
                        {
                            rawAddresses.Add(addr.Trim());
                        }
                    }
                }
            }
        }
        catch { }
        return rawAddresses;
    }

    /// <summary>
    /// 5. Добавить новый IP-адрес в список файрвола (Аналог add в CLI)
    /// </summary>
    /// <summary>
    /// Добавить новый IP-адрес в список файрвола с опциональным временем жизни (timeout)
    /// </summary>
    public async Task<bool> AddAddressAsync(string address, string listName, string? timeout = null)
    {

```

```

try
{
    // Создаем динамический объект для JSON payload
    object payload;
    if (!string.IsNullOrEmpty(timeout))
    {
        payload = new { address = address, list = listName, timeout = timeout };
    }
    else
    {
        payload = new { address = address, list = listName };
    }

    var content = new System.Net.Http.StringContent(
        System.Text.Json.JsonSerializer.Serialize(payload),
        System.Text.Encoding.UTF8,
        "application/json"
    );

    var response = await _httpClient.PutAsync("ip/firewall/address-list", content);
    return response.IsSuccessStatusCode;
}
catch
{
    return false;
}
}

/// <summary>
/// Проверка связи с роутером и аутентификация прав пользователя
/// </summary>
public async Task<ConnectionStatus> TestConnectionAsync()
{
    try
    {
        // Добавляем .ConfigureAwait(false) в конец асинхронного вызова
        var response = await _httpClient.GetAsync("system/resource").ConfigureAwait(false);

        if (response.IsSuccessStatusCode) return ConnectionStatus.Success;
        if (response.StatusCode == HttpStatusCode.Unauthorized) return ConnectionStatus.Unauthorized;

        return ConnectionStatus.Unreachable;
    }
    catch (HttpRequestException ex) when (ex.StatusCode == HttpStatusCode.Unauthorized)
    {
        return ConnectionStatus.Unauthorized;
    }
    catch
    {
        return ConnectionStatus.Unreachable;
    }
}

/// <summary>
/// 4. Запрос содержимого списка. Возвращает Словарь [IP-адрес -> Оставшийся таймаут]
/// </summary>
public async Task<Dictionary<string, string>> GetActiveAddressesWithTimeoutsAsync(string listName)
{
    var addressMap = new Dictionary<string, string>();
    try
    {
        // Явно просим Mikrotik вернуть address, disabled и timeout
        var response = await _httpClient.GetAsync($"ip/firewall/address-list?list={listName}&.proplist=address,disabled,timeout").ConfigureAwait(false);
        if (!response.IsSuccessStatusCode) return addressMap;

        var json = await response.Content.ReadAsStringAsync().ConfigureAwait(false);
        using (JsonDocument doc = JsonDocument.Parse(json))
        {
            if (doc.RootElement.ValueKind == JsonValueKind.Array)
            {
                foreach (var element in doc.RootElement.EnumerateArray())

```

```

{
    bool isDisabled = false;
    if (element.TryGetProperty("disabled", out JsonElement disabledProp))
    {
        string? disabledStr = disabledProp.GetString();
        if (!string.IsNullOrEmpty(disabledStr) && disabledStr.Equals("true",
            StringComparison.OrdinalIgnoreCase))
        {
            isDisabled = true;
        }
    }

    if (!isDisabled && element.TryGetProperty("address", out JsonElement addrProp))
    {
        string? addr = addrProp.GetString()?.Trim();
        if (!string.IsNullOrEmpty(addr))
        {
            // Забираем таймаут. Если его нет (статическая запись), запишем пустую строку
            string timeout = string.Empty;
            if (element.TryGetProperty("timeout", out JsonElement timeoutProp))
            {
                timeout = timeoutProp.GetString() ?? string.Empty;
            }

            // Складываем в словарь. Если Mikrotik вернул дубль (такое бывает при сбоях), перезапишем свежим
            addressMap[addr] = timeout;
        }
    }
}
catch { }
return addressMap;
}
}

```

```
[global::System.Configuration.UserScopedSettingAttribute()]
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
[global::System.Configuration.DefaultSettingValueAttribute("")]
public string EDLogFolder {
    get {
        return ((string)(this["EDLogFolder"]));
    }
    set {
        this["EDLogFolder"] = value;
    }
}
```

```
[global::System.Configuration.UserScopedSettingAttribute()]
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
[global::System.Configuration.DefaultSettingValueAttribute("")]
public string OutputFolder {
    get {
        return ((string)(this["OutputFolder"]));
    }
    set {
        this["OutputFolder"] = value;
    }
}
```

```
[global::System.Configuration.UserScopedSettingAttribute()]
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
[global::System.Configuration.DefaultSettingValueAttribute("")]
public string DataFolder {
    get {
        return ((string)(this["DataFolder"]));
    }
    set {
        this["DataFolder"] = value;
    }
}
```

```
[global::System.Configuration.UserScopedSettingAttribute()]
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
[global::System.Configuration.DefaultSettingValueAttribute("")]
public string PrivateKey {
    get {
        return ((string)(this["PrivateKey"]));
    }
    set {
        this["PrivateKey"] = value;
    }
}
```

```
[global::System.Configuration.UserScopedSettingAttribute()]
[global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
[global::System.Configuration.DefaultSettingValueAttribute("True")]
public bool AutoMonitoringEnabled {
    get {
        return ((bool)(this["AutoMonitoringEnabled"]));
    }
    set {
        this["AutoMonitoringEnabled"] = value;
    }
}
}
```

```
---Properties\Settings.settings---
<?xml version='1.0' encoding='utf-8'?>
<SettingsFile xmlns="http://schemas.microsoft.com/VisualStudio/2004/01/settings"
CurrentProfile="(Default)" GeneratedClassNamespace="EDIPSearch.Properties"
GeneratedClassName="Settings">
<Profiles />
<Settings>
<Setting Name="EDLogFolder" Type="System.String" Scope="User">
<Value Profile="(Default)" />
</Setting>
```

```

<Setting Name="OutputFolder" Type="System.String" Scope="User">
<Value Profile="(Default)" />
</Setting>
<Setting Name="DataFolder" Type="System.String" Scope="User">
<Value Profile="(Default)" />
</Setting>
<Setting Name="PrivateKey" Type="System.String" Scope="User">
<Value Profile="(Default)" />
</Setting>
<Setting Name="AutoMonitoringEnabled" Type="System.Boolean" Scope="User">
<Value Profile="(Default)">True</Value>
</Setting>
</Settings>
</SettingsFile>

```

```

---App.config---
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
<configSections>
<sectionGroup name="userSettings" type="System.Configuration.UserSettingsGroup, System,
Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089" >
<section name="EDIPSearch.Properties.Settings" type="System.Configuration.ClientSettingsSection,
System, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089"
allowExeDefinition="MachineToLocalUser" requirePermission="false" />
</sectionGroup>
</configSections>
<userSettings>
<EDIPSearch.Properties.Settings>
<setting name="EDLogFolder" serializeAs="String">
<value />
</setting>
<setting name="OutputFolder" serializeAs="String">
<value />
</setting>
<setting name="DataFolder" serializeAs="String">
<value />
</setting>
<setting name="PrivateKey" serializeAs="String">
<value />
</setting>
<setting name="AutoMonitoringEnabled" serializeAs="String">
<value>True</value>
</setting>
</EDIPSearch.Properties.Settings>
</userSettings>
</configuration>

```

```

---App.xaml---
<Application x:Class="EDIPSearch.App"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="clr-namespace:EDIPSearch"
StartupUri="MainWindow.xaml">
<Application.Resources>

</Application.Resources>
</Application>

```

```

---App.xaml.cs---
using System.Configuration;
using System.Data;
using System.Windows;

namespace EDIPSearch
{
    /// <summary>
    /// Interaction logic for App.xaml
    /// </summary>
    public partial class App : Application
    {
    }
}

```

```
}
```

```
---AssemblyInfo.cs---
```

```
using System.Windows;
```

```
[assembly: ThemeInfo(  
ResourceDictionaryLocation.None, //where theme specific resource dictionaries are located  
// (used if a resource is not found in the page,  
// or application resource dictionaries)  
ResourceDictionaryLocation.SourceAssembly //where the generic resource dictionary is located  
// (used if a resource is not found in the page,  
// app, or any theme specific resource dictionaries)  
)]
```

```
---EliteTheme.xaml---
```

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
```

```
<!-- 1. АУТЕНТИЧНЫЙ И УПРАВЛЯЕМЫЙ СТИЛЬ КНОПОК -->
```

```
<Style x:Key="EliteButton" TargetType="Button">  
<Setter Property="Background" Value="#1B2228"/>  
<Setter Property="Foreground" Value="#FF7100"/>  
<Setter Property="BorderBrush" Value="#FF7100"/>  
<Setter Property="BorderThickness" Value="1"/>  
<Setter Property="Padding" Value="12,5"/>  
<Setter Property="FontWeight" Value="Bold"/>  
<Setter Property="Template">  
<Setter.Value>  
<ControlTemplate TargetType="Button">  
<Border x:Name="border"  
Background="{TemplateBinding Background}"  
BorderBrush="{TemplateBinding BorderBrush}"  
BorderThickness="{TemplateBinding BorderThickness}"  
CornerRadius="2"  
Padding="{TemplateBinding Padding}">  
<ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center"/>  
</Border>  
<ControlTemplate.Triggers>  
<Trigger Property="IsMouseOver" Value="True">  
<Setter TargetName="border" Property="Background" Value="#33FF7100"/>  
<Setter Property="Foreground" Value="#FFA000"/>  
<Setter Property="BorderBrush" Value="#FFA000"/>  
</Trigger>  
<Trigger Property="IsPressed" Value="True">  
<Setter TargetName="border" Property="Background" Value="#55FF7100"/>  
<Setter Property="Foreground" Value="#FFFFFF"/>  
</Trigger>  
<Trigger Property="IsEnabled" Value="False">  
<Setter TargetName="border" Property="Background" Value="#0D1115"/>  
<Setter TargetName="border" Property="BorderBrush" Value="#44FF7100"/>  
<Setter Property="Foreground" Value="#44FF7100"/>  
</Trigger>  
</ControlTemplate.Triggers>  
</ControlTemplate>  
</Setter.Value>  
</Setter>  
</Style>
```

```
<!-- 2. СТИЛЬ ДЛЯ CHECKBOX (Оранжевый контур и темный фон) -->
```

```
<Style TargetType="CheckBox">  
<Setter Property="Foreground" Value="#E0E6ED"/>  
<Setter Property="FontWeight" Value="Bold"/>  
<Setter Property="VerticalAlignment" Value="Center"/>  
<Setter Property="Template">  
<Setter.Value>  
<ControlTemplate TargetType="CheckBox">  
<BulletDecorator Background="Transparent" Cursor="Hand">  
<BulletDecorator.Bullet>  
<Border x:Name="border" Width="14" Height="14" Background="#11161B" BorderBrush="#FF7100"  
BorderThickness="1" CornerRadius="1">  
<Grid x:Name="checkmark" Visibility="Collapsed">
```

```

<Path Data="M 2,6 L 5,9 L 10,2" Stroke="#FFA000" StrokeThickness="2" StrokeStartLineCap="Round"
StrokeEndLineCap="Round"/>
</Grid>
</Border>
</BulletDecorator.Bullet>
<ContentPresenter Margin="6,0,0,0" VerticalAlignment="Center"/>
</BulletDecorator>
<ControlTemplate.Triggers>
<Trigger Property="IsChecked" Value="True">
<Setter TargetName="checkmark" Property="Visibility" Value="Visible"/>
<Setter TargetName="border" Property="Background" Value="#1B2228"/>
</Trigger>
<Trigger Property="IsMouseOver" Value="True">
<Setter TargetName="border" Property="BorderBrush" Value="#FFA000"/>
<Setter TargetName="border" Property="Background" Value="#22FF7100"/>
</Trigger>
<Trigger Property="IsEnabled" Value="False">
<Setter TargetName="border" Property="BorderBrush" Value="#44FF7100"/>
<Setter Property="Foreground" Value="#44FF7100"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>
<!-- 3. СТИЛЬ ДЛЯ TEXTBOX (Поля ввода теперь тёмные с оранжевым контуром) -->
<Style TargetType="TextBox">
<Setter Property="Background" Value="#11161B"/>
<Setter Property="Foreground" Value="#E0E6ED"/>
<Setter Property="BorderBrush" Value="#33FF7100"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="Padding" Value="5,3"/>
<Setter Property="CaretBrush" Value="#FF7100"/>
<!-- Оранжевый курсор ввода -->
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="TextBox">
<Border x:Name="border"
Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}"
BorderThickness="{TemplateBinding BorderThickness}"
CornerRadius="2">
<!-- Внутренний контейнер для текста WPF -->
<ScrollViewer x:Name="PART_ContentHost" Focusable="false" HorizontalScrollBarVisibility="Hidden"
VerticalScrollBarVisibility="Hidden"/>
</Border>
<ControlTemplate.Triggers>
<!-- При фокусе (клике внутрь) рамка ярко загорается -->
<Trigger Property="IsFocused" Value="True">
<Setter TargetName="border" Property="BorderBrush" Value="#FF7100"/>
<Setter Property="Background" Value="#161C20"/>
</Trigger>
<Trigger Property="IsMouseOver" Value="True">
<Setter TargetName="border" Property="BorderBrush" Value="#FFA000"/>
</Trigger>
<Trigger Property="IsEnabled" Value="False">
<Setter Property="Foreground" Value="#44FF7100"/>
<Setter TargetName="border" Property="BorderBrush" Value="#22FF7100"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- Мапим стиль для PasswordBox (для окон настроек) -->
<!-- ИСПРАВЛЕННЫЙ СТИЛЬ ДЛЯ PASSWORDBOX -->
<Style TargetType="PasswordBox">
<Setter Property="Background" Value="#11161B"/>
<Setter Property="Foreground" Value="#E0E6ED"/>
<Setter Property="BorderBrush" Value="#33FF7100"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="Padding" Value="5,3"/>
<Setter Property="CaretBrush" Value="#FF7100"/>
<Setter Property="Template">
<Setter.Value>

```

```
<ControlTemplate TargetType="PasswordBox">
<Border x:Name="border"
Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}"
BorderThickness="{TemplateBinding BorderThickness}"
CornerRadius="2">
<!-- Для PasswordBox внутренний контейнер называется PART_ContentHost, как и у TextBox -->
<ScrollViewer x:Name="PART_ContentHost" Focusable="false" HorizontalScrollBarVisibility="Hidden"
VerticalScrollBarVisibility="Hidden"/>
</Border>
<ControlTemplate.Triggers>
<Trigger Property="IsFocused" Value="True">
<Setter TargetName="border" Property="BorderBrush" Value="#FF7100"/>
<Setter Property="Background" Value="#161C20"/>
</Trigger>
<Trigger Property="IsMouseOver" Value="True">
<Setter TargetName="border" Property="BorderBrush" Value="#FFA000"/>
</Trigger>
<Trigger Property="IsEnabled" Value="False">
<Setter Property="Foreground" Value="#44FF7100"/>
<Setter TargetName="border" Property="BorderBrush" Value="#22FF7100"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>
```

```
<!-- 4. СТИЛЬ ДЛЯ TOOLBAR (Панели инструментов) -->
<Style TargetType="ToolBar">
<Setter Property="Background" Value="#1B2228"/>
<Setter Property="BorderBrush" Value="#33FF7100"/>
<Setter Property="BorderThickness" Value="0,0,0,1"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ToolBar">
<Border Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}"
BorderThickness="{TemplateBinding BorderThickness}"
Padding="5">
<!-- Выводим элементы панели внутри Grid -->
<ToolBarPanel x:Name="PART_ToolBarPanel" IsItemsHost="true" Orientation="Horizontal"/>
</Border>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>
```

```
<!-- Перекрашиваем контейнер панелей ToolBarTray -->
<Style TargetType="ToolBarTray">
<Setter Property="Background" Value="#11161B"/>
</Style>
```

```
<!-- 5. СТИЛИ ДЛЯ TABCONTROL И TABITEM -->
<Style TargetType="TabControl">
<Setter Property="Background" Value="#1B2228"/>
<Setter Property="BorderBrush" Value="#33FF7100"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="Padding" Value="10"/>
</Style>
```

```
<Style TargetType="TabItem">
<Setter Property="Background" Value="#11161B"/>
<Setter Property="Foreground" Value="#FF7100"/>
<Setter Property="BorderBrush" Value="#33FF7100"/>
<Setter Property="BorderThickness" Value="1,1,1,0"/>
<Setter Property="Padding" Value="15,6"/>
<Setter Property="FontWeight" Value="Bold"/>
<Setter Property="Margin" Value="0,0,2,0"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="TabItem">
```

```

<Border x:Name="Panel" Background="{TemplateBinding Background}" BorderBrush="{TemplateBinding
BorderBrush}" BorderThickness="{TemplateBinding BorderThickness}" CornerRadius="3,3,0,0"
Padding="{TemplateBinding Padding}" Cursor="Hand">
<ContentPresenter x:Name="ContentSite" ContentSource="Header" HorizontalAlignment="Center"
VerticalAlignment="Center"/>
</Border>
<ControlTemplate.Triggers>
<Trigger Property="IsSelected" Value="True">
<Setter TargetName="Panel" Property="Background" Value="#1B2228"/>
<Setter Property="Foreground" Value="#FFA000"/>
<Setter TargetName="Panel" Property="BorderBrush" Value="#FF7100"/>
<Setter TargetName="Panel" Property="BorderThickness" Value="1,2,1,0"/>
</Trigger>
<MultiTrigger>
<MultiTrigger.Conditions>
<Condition Property="IsMouseOver" Value="True"/>
<Condition Property="IsSelected" Value="False"/>
</MultiTrigger.Conditions>
<Setter TargetName="Panel" Property="Background" Value="#22FF7100"/>
<Setter Property="Foreground" Value="#FFA000"/>
</MultiTrigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

```

```

<!-- 6. ЖЕЛЕЗОВЕТОННАЯ СТИЛИЗАЦИЯ LISTBOX С ОРАНЖЕВЫМ СКРОЛЛОМ -->
<Style TargetType="ListBox">
<Setter Property="Background" Value="#11161B"/>
<Setter Property="Foreground" Value="#FFA000"/>
<Setter Property="BorderBrush" Value="#33FF7100"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="ScrollViewer.HorizontalScrollBarVisibility" Value="Disabled"/>
<!-- Отключаем горизонтальную полосу -->
<Setter Property="ScrollViewer.VerticalScrollBarVisibility" Value="Auto"/>

```

```

<!-- НАПРАВЛЯЕМ ПЕРЕНОС СТРОК НА ВСЕ ВНУТРЕННИЕ СТИЛИ СРАЗУ -->
<Style.Resources>
<!-- Стили ползунка (ScrollThumbs) -->
<Style x:Key="ScrollThumbs" TargetType="Thumb">
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="Thumb">
<Grid x:Name="Grid">
<Border HorizontalAlignment="Stretch" VerticalAlignment="Stretch"
Background="#55FF7100" CornerRadius="3" MinHeight="20" />
</Grid>
<ControlTemplate.Triggers>
<Trigger Property="IsMouseOver" Value="true">
<Setter Property="Background" Value="#FF7100"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

```

```

<!-- Стили Скроллбара -->
<Style TargetType="ScrollBar">
<Setter Property="Background" Value="#11161B" />
<Setter Property="Width" Value="8" />
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ScrollBar">
<Grid x:Name="Bg" Background="#11161B" SnapsToDevicePixels="true">
<Track x:Name="PART_Track" IsDirectionReversed="true" IsEnabled="{TemplateBinding IsEnabled}">
<Track.DecreaseRepeatButton>
<RepeatButton Command="ScrollBar.LineUpCommand" Opacity="0" Focusable="false" />
</Track.DecreaseRepeatButton>
<Track.Thumb>
<Thumb Style="{StaticResource ScrollThumbs}" />
</Track.Thumb>
<Track.IncreaseRepeatButton>
<RepeatButton Command="ScrollBar.LineDownCommand" Opacity="0" Focusable="false" />

```

```
</Track.IncreaseRepeatButton>
</Track>
</Grid>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>
</Style.Resources>
</Style>
```

```
<!-- 7. ФИНАЛЬНЫЙ СТИЛЬ ДЛЯ СТРОК СПИСКА С ПОВЫШЕННОЙ ЯРКОСТЬЮ НЕОНА -->
<Style TargetType="ListBoxItem">
<Setter Property="Background" Value="Transparent"/>
<Setter Property="Foreground" Value="#FFA000"/>
<Setter Property="Padding" Value="8,6"/>
<Setter Property="Margin" Value="0,0,0,2"/>
<Setter Property="HorizontalAlignment" Value="Stretch"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ListBoxItem">
<!-- 1. ДЕФОЛТНОЕ СОСТОЯНИЕ: Сделали нижнюю линию ярче (#33 вместо #19) -->
<Border x:Name="IconBorder"
Background="{TemplateBinding Background}"
Padding="{TemplateBinding Padding}"
CornerRadius="2"
BorderBrush="#33FF7100"
BorderThickness="0,0,0,1">
```

```
<TextBlock Text="{Binding .}"
TextWrapping="Wrap"
Foreground="{TemplateBinding Foreground}"
FontSize="11"/>
</Border>
<ControlTemplate.Triggers>
<!-- 2. НАВЕДЕНИЕ МЫШИ (MOUSE OVER): Сделали фон и линию ощутимо сочнее -->
<Trigger Property="IsMouseOver" Value="true">
<!-- Фон стал плотнее (#25 вместо #1A) -->
<Setter TargetName="IconBorder" Property="Background" Value="#25FF7100"/>
<!-- Нижняя линия разгорается в полную силу (#66 вместо #44) -->
<Setter TargetName="IconBorder" Property="BorderBrush" Value="#66FF7100"/>
<Setter Property="Foreground" Value="FFFFFF"/>
</Trigger>
<!-- 3. СТРОКА ВЫДЕЛЕНА КЛИКОМ (SELECTED) -->
<Trigger Property="IsSelected" Value="true">
<Setter TargetName="IconBorder" Property="Background" Value="#44FF7100"/>
<Setter TargetName="IconBorder" Property="BorderBrush" Value="#FF7100"/>
<Setter Property="Foreground" Value="FFFFFF"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>
```

```
<!-- 9. ПРАВИЛЬНАЯ СТИЛИЗАЦИЯ LISTVIEW (Сохраняет колонки GridView и раскрашивает строки) -->
<Style TargetType="ListViewItem">
<Setter Property="Background" Value="Transparent"/>
<Setter Property="Foreground" Value="#FFA000"/>
<Setter Property="Padding" Value="6,4"/>
<Setter Property="HorizontalAlignment" Value="Stretch"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ListViewItem">
<!-- Задаем строку таблицы с нижней тонкой неоновой линией -->
<Border x:Name="RowBorder"
Background="{TemplateBinding Background}"
BorderBrush="#19FF7100"
BorderThickness="0,0,0,1"
Padding="{TemplateBinding Padding}">
```

```
<!-- ИСПРАВЛЕННЫЙ ЭЛЕМЕНТ: Отвечает за правильный вывод колонок GridView в WPF -->
```

```

<GridViewRowPresenter Content="{TemplateBinding Content}"/>

</Border>
<!-- Ниже идут твои ControlTemplate.Triggers без изменений -->

<ControlTemplate.Triggers>
<Trigger Property="IsMouseOver" Value="true">
<Setter TargetName="RowBorder" Property="Background" Value="#1AFF7100"/>
<Setter Property="Foreground" Value="#FFFFFF"/>
</Trigger>
<Trigger Property="IsSelected" Value="true">
<Setter TargetName="RowBorder" Property="Background" Value="#33FF7100"/>
<Setter TargetName="RowBorder" Property="BorderBrush" Value="#FF7100"/>
<Setter Property="Foreground" Value="#FFFFFF"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- 7. ИСПРАВЛЕННАЯ КНОПКА-СТРЕЛКА ДЛЯ СОМВОХОВ (Зафиксировали справа) -->
<Style x:Key="ComboBoxToggleButton" TargetType="ToggleButton">
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ToggleButton">
<!-- ДОБАВЛЕНО: Прижимаем к правому краю и ставим ширину 24 -->
<Border x:Name="Border"
Background="#1B2228"
BorderBrush="#33FF7100"
BorderThickness="0,1,1,1"
CornerRadius="0,2,2,0"
HorizontalAlignment="Right"
Width="24"
Height="{TemplateBinding Height}">
<!-- Рисуем оранжевый треугольник-стрелочку -->
<Path x:Name="Arrow"
Data="M 0 0 L 4 4 L 8 0 Z"
Fill="#FF7100"
HorizontalAlignment="Center"
VerticalAlignment="Center"/>
</Border>
<ControlTemplate.Triggers>
<Trigger Property="IsMouseOver" Value="true">
<Setter TargetName="Border" Property="Background" Value="#33FF7100"/>
<Setter TargetName="Arrow" Property="Fill" Value="#FFA000"/>
</Trigger>
<Trigger Property="IsChecked" Value="true">
<Setter TargetName="Border" Property="Background" Value="#55FF7100"/>
<Setter TargetName="Arrow" Property="Fill" Value="#FFFFFF"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- [ШАГ 2] ТОЛЬКО ТЕПЕРЬ ОБЪЯВЛЯЕМ САМ СОМВОХОВ, ИСПОЛЬЗУЮЩИЙ СТИЛЬ ВЫШЕ -->
<Style TargetType="ComboBox">
<Setter Property="Background" Value="#11161B"/>
<Setter Property="Foreground" Value="#E0E6ED"/>
<Setter Property="BorderBrush" Value="#33FF7100"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="Padding" Value="5,3"/>
<Setter Property="SnapsToDevicePixels" Value="true"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ComboBox">
<Grid x:Name="MainGrid">

```

```

<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="24" />
</Grid.ColumnDefinitions>

<!-- Кнопка-стрелка раскрытия (вызывает стиль по ключу сверху) -->
<ToggleButton Style="{StaticResource ComboBoxToggleButton}"
Grid.Column="0" Grid.ColumnSpan="2"
Focusable="false"
IsChecked="{Binding IsDropDownOpen, Mode=TwoWay, RelativeSource={RelativeSource
TemplatedParent}}"/>

<!-- Фоновая рамка текстовой зоны -->
<Border x:Name="ContentBorder" Background="#11161B" BorderBrush="#33FF7100"
BorderThickness="1,1,0,1" CornerRadius="2,0,0,2" Margin="0"/>

<!-- Контейнер для статического текста -->
<ContentPresenter Name="ContentSite" IsHitTestVisible="False"
Content="{TemplateBinding SelectionBoxItem}"
ContentTemplate="{TemplateBinding SelectionBoxItemTemplate}"
ContentTemplateSelector="{TemplateBinding ItemTemplateSelector}"
Margin="6,3,0,3" VerticalAlignment="Center" HorizontalAlignment="Left"/>

<!-- Внутренний TextBox для режима IsEditable="True" -->
<TextBox x:Name="PART_EditableTextBox"
HorizontalAlignment="Stretch" VerticalAlignment="Center"
Margin="3,2,2,2" Focusable="true"
Background="Transparent" Foreground="#E0E6ED"
BorderThickness="0" Visibility="Hidden"
IsReadOnly="{TemplateBinding IsReadOnly}">
<TextBox.Style>
<Style TargetType="TextBox">
<Setter Property="CaretBrush" Value="#FF7100"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="TextBox">
<ScrollViewer x:Name="PART_ContentHost" Focusable="false" Background="Transparent"/>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>
</TextBox.Style>
</TextBox>

<!-- Выпадающее окно со списком -->
<Popup Name="Popup" Placement="Bottom" IsOpen="{TemplateBinding IsDropDownOpen}"
AllowsTransparency="true" Focusable="false" PopupAnimation="Slide">
<Grid Name="DropDown" SnapsToDevicePixels="true" MinWidth="{TemplateBinding ActualWidth}"
MaxHeight="{TemplateBinding MaxDropDownHeight}">
<Border Name="DropDownBorder" Background="#11161B" BorderBrush="#FF7100" BorderThickness="1"
CornerRadius="0,0,2,2"/>
<ScrollViewer Margin="4,6,4,6" SnapsToDevicePixels="true">
<StackPanel IsItemsHost="true" KeyboardNavigation.DirectionalNavigation="Contained"/>
</ScrollViewer>
</Grid>
</Popup>
</Grid>
<ControlTemplate.Triggers>
<Trigger Property="IsEditable" Value="true">
<Setter Property="IsTabStop" Value="false"/>
<Setter TargetName="PART_EditableTextBox" Property="Visibility" Value="Visible"/>
<Setter TargetName="ContentSite" Property="Visibility" Value="Hidden"/>
</Trigger>
<Trigger Property="IsMouseOver" Value="true">
<Setter TargetName="ContentBorder" Property="BorderBrush" Value="#FFA000"/>
</Trigger>
<Trigger Property="IsFocused" Value="true">
<Setter TargetName="ContentBorder" Property="BorderBrush" Value="#FF7100"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

```

```

<!-- 9. СТИЛИЗАЦИЯ СТРОК ВНУТРИ ВЫПАДАЮЩЕГО СПИСКА СОМБОВОХ -->
<Style TargetType="ComboBoxItem">
<Setter Property="Background" Value="Transparent"/>
<Setter Property="Foreground" Value="#FFA000"/>
<Setter Property="Padding" Value="6,4"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ComboBoxItem">
<Border x:Name="ItemBorder" Background="{TemplateBinding Background}" Padding="{TemplateBinding
Padding}" CornerRadius="2">
<ContentPresenter />
</Border>
<ControlTemplate.Triggers>
<Trigger Property="IsMouseOver" Value="true">
<Setter TargetName="ItemBorder" Property="Background" Value="#1AFF7100"/>
<Setter Property="Foreground" Value="#FFFFFF"/>
</Trigger>
<Trigger Property="IsHighlighted" Value="true">
<Setter TargetName="ItemBorder" Property="Background" Value="#33FF7100"/>
<Setter Property="Foreground" Value="#FFFFFF"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- ДОБАВЛЕНО: Принудительно заставляем TextBox внутри ToolBar использовать нашу тему -->
<Style x:Key="{x:Static ToolBar.TextBoxStyleKey}" TargetType="TextBox" BasedOn="{StaticResource
{x:Type TextBox}}">
<Setter Property="Width" Value="120"/>
<!-- Задай базовую ширину поля на тулбаре, если нужно -->
<Setter Property="Margin" Value="2,0"/>
</Style>
<!-- ИСПРАВЛЕННЫЙ СТИЛЬ ДЛЯ LABEL (Убрали жесткие margins, чтобы не ломать старую верстку) -->
<Style TargetType="Label">
<Setter Property="Foreground" Value="#AAFF7100"/>
<!-- Наш фирменный приглушенный оранжевый -->
<Setter Property="FontWeight" Value="SemiBold"/>
<!-- Мы убираем отсюда VerticalAlignment и Margin.
Для новых окон (frmMikrotikRestConfig) мы пропишем выравнивание прямо в XAML самого окна,
чтобы это никак не задевало твою старую форму настроек фильтров! -->
</Style>
<!-- 9. СТИЛЬ ДЛЯ TEXTBLOCK (Окрашиваем весь простой текст в читаемый цвет) -->
<Style TargetType="TextBlock">
<!-- Делаем базовый текст светло-серым, чтобы знаки + и - стали видны -->
<Setter Property="Foreground" Value="#E0E6ED"/>
<!-- Задаем сглаживание шрифтов для идеального отображения на темном фоне -->
<Setter Property="TextOptions.TextFormattingMode" Value="Display"/>
<Setter Property="TextOptions.TextRenderingMode" Value="ClearType"/>
</Style>

<!-- 9. ИСПРАВЛЕННАЯ СТИЛИЗАЦИЯ LISTVIEW (Оранжевый скроллбар и отключение горизонтальной полосы) -
-->
<Style TargetType="ListView">
<Setter Property="Background" Value="#11161B"/>
<Setter Property="Foreground" Value="#E0E6ED"/>
<Setter Property="BorderBrush" Value="#33FF7100"/>
<Setter Property="BorderThickness" Value="1"/>
<!-- Жестко отключаем горизонтальный скролл, чтобы убрать полосу снизу -->
<Setter Property="ScrollViewer.HorizontalScrollBarVisibility" Value="Disabled"/>
<Setter Property="ScrollViewer.VerticalScrollBarVisibility" Value="Auto"/>

<Style.Resources>
<!-- 1. СНАЧАЛА ЗАКИДЫВАЕМ СТИЛЬ ПОЛЗУНКА ВНУТРЬ РЕСУРСОВ LISTVIEW -->
<Style x:Key="ScrollThumbs" TargetType="Thumb">
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="Thumb">
<Grid x:Name="Grid">
<Border HorizontalAlignment="Stretch" VerticalAlignment="Stretch"

```

```

Background="#55FF7100" CornerRadius="3" MinHeight="20" />
</Grid>
<ControlTemplate.Triggers>
<Trigger Property="IsMouseOver" Value="true">
<Setter Property="Background" Value="#FF7100"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- 2. НАШ СКРОЛЛБАР С ИДЕАЛЬНО ТАКТИЧЕСКИ ВЫРОВНЕННЫМИ ТЕГАМИ -->
<Style TargetType="ScrollBar">
<Setter Property="Background" Value="#11161B" />
<Setter Property="Width" Value="8" />
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ScrollBar">
<Grid x:Name="Bg" Background="#11161B" SnapsToDevicePixels="true">
<!-- Track открывается -->
<Track x:Name="PART_Track" IsDirectionReversed="true" IsEnabled="{TemplateBinding IsEnabled}">
<Track.DecreaseRepeatButton>
<RepeatButton Command="ScrollBar.LineUpCommand" Opacity="0" Focusable="false" />
</Track.DecreaseRepeatButton>
<Track.Thumb>
<Thumb Style="{StaticResource ScrollThumbs}" />
</Track.Thumb>
<Track.IncreaseRepeatButton>
<RepeatButton Command="ScrollBar.LineDownCommand" Opacity="0" Focusable="false" />
</Track.IncreaseRepeatButton>
</Track>
<!-- Track закрывается строго на своем месте! -->
</Grid>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>
</Style.Resources>

</Style>
<!-- 10. ФИНАЛЬНАЯ ИЗОЛЯЦИЯ МЕНЮ (ПОЛНОСТЬЮ ВЫРЕЗАЕТ БЕЛЫЙ АРТЕФАКТ ТЕНИ WINDOWS) -->
<Style TargetType="ContextMenu">
<Setter Property="Background" Value="#1B2228"/>
<Setter Property="BorderBrush" Value="#FF7100"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="OverridesDefaultStyle" Value="True"/>
<Setter Property="Grid.IsSharedSizeScope" Value="True"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ContextMenu">
<!-- ГЛАВНЫЙ ФИКС: Мы полностью убрали SystemDropShadowChrome из шаблона!
Теперь здесь только наш чистый космический Border, и белому пятну физически неоткуда взяться. -->
<Border Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}"
BorderThickness="{TemplateBinding BorderThickness}"
CornerRadius="2"
Padding="2">
<!-- Внутренний контейнер, который выводит пункты MenuItem строго вертикально -->
<StackPanel IsItemsHost="True" KeyboardNavigation.DirectionalNavigation="Cycle"/>
</Border>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- СТИЛИЗАЦИЯ ПУНКТОВ МЕНЮ С ЯРКИМ ЧИТАЕМЫМ ТЕКСТОМ -->
<Style TargetType="MenuItem">
<Setter Property="Background" Value="Transparent"/>
<Setter Property="Foreground" Value="#FF7100"/>
<!-- Наш фирменный оранжевый текст -->
<Setter Property="FontWeight" Value="Bold"/>
<Setter Property="Padding" Value="15,8"/>
<Setter Property="OverridesDefaultStyle" Value="True"/>
<Setter Property="Template">

```

```

<Setter.Value>
<ControlTemplate TargetType="MenuItem">
<Border x:Name="MenuBorder"
Background="{TemplateBinding Background}"
CornerRadius="2"
Margin="1"
Padding="{TemplateBinding Padding}">

<!-- Выводим текст. Принудительно ставим контрастный цвет Foreground -->
<TextBlock Text="{TemplateBinding Header}"
Foreground="{TemplateBinding Foreground}"
VerticalAlignment="Center"
HorizontalAlignment="Left"
FontSize="11"/>
</Border>
<ControlTemplate.Triggers>
<!-- ЭФФЕКТ НАВЕДЕНИЯ МЫШИ (HOVER) -->
<Trigger Property="IsMouseOver" Value="True">
<Setter TargetName="MenuBorder" Property="Background" Value="#33FF7100"/>
<Setter Property="Foreground" Value="FFFFFF"/>
<!-- При наведении текст горит белым неонам -->
</Trigger>
<!-- Дублируем для триггера фокуса таблицы -->
<Trigger Property="IsHighlighted" Value="True">
<Setter TargetName="MenuBorder" Property="Background" Value="#33FF7100"/>
<Setter Property="Foreground" Value="FFFFFF"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

</ResourceDictionary>

---frmMikrotikRestConfig.xaml---
<Window x:Class="EDIPSearch.frmMikrotikRestConfig"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:local="clr-namespace:EDIPSearch"
mc:Ignorable="d"
Title="Интеграция: Настройка маршрутизаторов Mikrotik Rest API" Height="420" Width="680"
WindowStartupLocation="CenterOwner" Background="#11161B" Foreground="#E0E6ED"
Icon="pack://application:,,,/app_icon.ico">
<Window.Resources>
<ResourceDictionary>
<ResourceDictionary.MergedDictionaries>
<!-- Подключаем созданный файл темы из ресурсов проекта -->
<ResourceDictionary Source="EliteTheme.xaml" />
</ResourceDictionary.MergedDictionaries>
</ResourceDictionary>
</Window.Resources>
<Grid Margin="10">

<Grid.ColumnDefinitions>
<ColumnDefinition Width="220"/>
<ColumnDefinition Width="*"/>
</Grid.ColumnDefinitions>

<!-- Левая панель: Список роутеров REST API -->
<Grid Grid.Column="0" Margin="0,0,10,0">
<Grid.RowDefinitions>
<RowDefinition Height="*"/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>
<ListBox x:Name="lstRouters" DisplayMemberPath="Name"
SelectionChanged="LstRouters_SelectionChanged"/>
<StackPanel Grid.Row="1" Orientation="Horizontal" HorizontalAlignment="Center" Margin="0,10,0,0">
<Button Content="Добавить" Width="85" Margin="5" Click="BtnAdd_Click" Background="{x:Null}"
Style="{StaticResource EliteButton}"/>

```

```

<Button Content="Удалить" Width="85" Margin="5" Click="BtnDelete_Click" Style="{StaticResource
EliteButton}"/>
</StackPanel>
</Grid>

<!-- Правая панель: Редактор выбранного роутера -->
<Border Grid.Column="1" Background="#FF1B1611" CornerRadius="5" Padding="15"
BorderBrush="#FFDA761D" BorderThickness="1">
<Grid x:Name="gridEditor" Visibility="Collapsed">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>
<Grid.ColumnDefinitions>
<ColumnDefinition Width="180"/>
<ColumnDefinition Width="*"/>
</Grid.ColumnDefinitions>

<Label Content="Название конфигурации:" Grid.Row="0" VerticalAlignment="Center"/>
<TextBox x:Name="txtName" Grid.Row="0" Grid.Column="1" Margin="5"/>

<Label Content="Внутренний IP (LAN):" Grid.Row="1" VerticalAlignment="Center"/>
<TextBox x:Name="txtIp" Grid.Row="1" Grid.Column="1" Margin="5"/>

<Grid Grid.Row="2" Grid.Column="0" Grid.ColumnSpan="2">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="130"/>
<ColumnDefinition Width="*"/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>
<Label Content="Порт REST API:" VerticalAlignment="Center"/>
<TextBox x:Name="txtPort" Grid.Column="1" Margin="5"/>
<CheckBox x:Name="chkSsl" Content="Использовать SSL" Grid.Column="2" VerticalAlignment="Center"
Margin="10,0,5,0"/>
</Grid>

<Label Content="Имя пользователя:" Grid.Row="3" VerticalAlignment="Center"/>
<TextBox x:Name="txtUser" Grid.Row="3" Grid.Column="1" Margin="5"/>

<Label Content="Пароль:" Grid.Row="4" VerticalAlignment="Center"/>
<PasswordBox x:Name="txtPass" Grid.Row="4" Grid.Column="1" Margin="5"
PasswordChanged="TxtPass_PasswordChanged"/>

<Label Content="Целевой список (ED):" Grid.Row="5" VerticalAlignment="Center"/>
<Grid Grid.Row="5" Grid.Column="1" Margin="5">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*"/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>
<!-- Выпадающий список, полученный прямо из микротика -->
<ComboBox x:Name="cmbAddressLists" IsEditable="True" ToolTip="Выберите из существующих или введите
новое имя"/>
<Button x:Name="btnFetchLists" Content="□ " Grid.Column="1" Width="30" Margin="5,0,0,0"
ToolTip="Загрузить доступные списки с роутера" Click="BtnFetchLists_Click" Style="{StaticResource
EliteButton}" />
</Grid>

<!-- Шаг 5 (Целевой список) у нас уже есть. Добавляем на Роу 6 (Внешний интерфейс): -->
<Label Content="Внешний (WAN) интерфейс:" Grid.Row="6" VerticalAlignment="Center"/>
<Grid Grid.Row="6" Grid.Column="1" Margin="5">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*"/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>
<!-- Заменили TextBox на ComboBox -->

```

```

<ComboBox x:Name="cmbWanInterface" IsEditable="True" ToolTip="Выберите WAN-интерфейс роутера или
введите вручную"/>
<Button x:Name="btnFetchInterfaces" Content="□" Grid.Column="1" Width="30" Margin="5,0,0,0"
ToolTip="Загрузить доступные интерфейсы" Click="BtnFetchInterfaces_Click" Style="{StaticResource
EliteButton}"/>
</Grid>

<!-- Сдвигаем индикатор проверки связи на Ров 7 -->
<StackPanel Grid.Row="7" Grid.Column="0" Grid.ColumnSpan="2" Orientation="Horizontal"
VerticalAlignment="Center" Margin="5,10,0,0">
<Button x:Name="btnTest" Content="Проверить связь" Width="130" Height="28" Click="BtnTest_Click"
Style="{StaticResource EliteButton}"/>
<TextBlock x:Name="lblTestStatus" VerticalAlignment="Center" Margin="15,0,0,0" FontWeight="Bold"/>
</StackPanel>

<!-- Сдвигаем сохранение на Ров 8 -->
<Button Content="Сохранить настройки" Grid.Row="8" Grid.Column="1" HorizontalAlignment="Right"
Width="140" Height="28" Click="BtnSave_Click" Style="{StaticResource EliteButton}"/>
</Grid>
</Border>
</Grid>
</Window>

```

```

---frmMikrotikRestConfig.xaml.cs---

```

```

using System;
using System.Collections.Generic;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using EDIPSearch.Models;
using EDIPSearch.Network;

namespace EDIPSearch
{
    public partial class frmMikrotikRestConfig : Window
    {
        private List<MikrotikConfig>? _routers;
        private MikrotikConfig? _selectedRouter;
        private bool _isBinding = false;

        public frmMikrotikRestConfig()
        {
            InitializeComponent();
            LoadData();
        }

        private void LoadData()
        {
            _routers = RouterStorage.Load();
            lstRouters.ItemsSource = null;
            lstRouters.ItemsSource = _routers;
        }

        private void LstRouters_SelectionChanged(object sender, SelectionChangedEventArgs e)
        {
            if (lstRouters.SelectedItem is MikrotikConfig router)
            {
                _isBinding = true;
                _selectedRouter = router;
                gridEditor.Visibility = Visibility.Visible;

                txtName.Text = router.Name;
                txtIp.Text = router.InternalIp;
                txtPort.Text = router.Port.ToString();
                chkSsl.IsChecked = router.UseSsl;
                txtUser.Text = router.Username;
                txtPass.Password = router.Password;
                cmbWanInterface.Text = router.WanInterface;
                cmbWanInterface.ItemsSource = null; // Сбрасываем старый список при переключении роутеров
            }
        }
    }
}

```

```

cmbAddressLists.Text = router.TargetAddressList;
cmbAddressLists.ItemsSource = null; // Очищаем старые варианты при смене роутера

lblTestStatus.Text = string.Empty;
_isBinding = false;
}
else
{
gridEditor.Visibility = Visibility.Collapsed;
}
}

private void BtnAdd_Click(object sender, RoutedEventArgs e)
{
var newRouter = new MikrotikConfig { Name = "Новый Mikrotik REST" };
if(_routers==null) _routers = new List<MikrotikConfig>();
_routers.Add(newRouter);
lstRouters.ItemsSource = null;
lstRouters.ItemsSource = _routers;
lstRouters.SelectedItem = newRouter;
}

private void BtnDelete_Click(object sender, RoutedEventArgs e)
{
if (lstRouters.SelectedItem is MikrotikConfig router)
{
if (_routers != null)
{
_routers.Remove(router);
lstRouters.ItemsSource = null;
lstRouters.ItemsSource = _routers;
gridEditor.Visibility = Visibility.Collapsed;
}
}
}

private void TxtPass_PasswordChanged(object sender, RoutedEventArgs e)
{
if (!_isBinding && _selectedRouter != null)
_selectedRouter.Password = txtPass.Password;
}

private void ApplyFormToModel()
{
if (_selectedRouter == null) return;
_selectedRouter.Name = txtName.Text;
_selectedRouter.InternalIp = txtIp.Text;
_selectedRouter.Port = int.TryParse(txtPort.Text, out int p) ? p : 443;
_selectedRouter.UseSsl = chkSsl.IsChecked ?? true;
_selectedRouter.Username = txtUser.Text;
_selectedRouter.Password = txtPass.Password;
_selectedRouter.TargetAddressList = cmbAddressLists.Text; // Поддерживает ручной ввод или выбор
// _selectedRouter.WanInterface = txtWanInterface.Text; // Сбор из UI
_selectedRouter.WanInterface = cmbWanInterface.Text;

}

private async void BtnFetchInterfaces_Click(object sender, RoutedEventArgs e)
{
if (_selectedRouter == null) return;
ApplyFormToModel();

btnFetchInterfaces.IsEnabled = false;
var client = new MikrotikRestClient(_selectedRouter);

// Запрашиваем реальный список интерфейсов с этого Mikrotik
var availableInterfaces = await client.GetInterfaceNamesAsync();

```

```

if (availableInterfaces.Count > 0)
{
    string currentSelection = cmbWanInterface.Text;
    cmbWanInterface.ItemsSource = availableInterfaces;
    cmbWanInterface.Text = currentSelection; // Сохраняем текущий выбор пользователя
}
else
{
    MessageBox.Show("Не удалось получить список интерфейсов. Проверьте настройки связи.", "Ошибка API",
    MessageBoxButton.OK, MessageBoxImage.Warning);
}
btnFetchInterfaces.IsEnabled = true;
}

private async void BtnFetchLists_Click(object sender, RoutedEventArgs e)
{
    if (_selectedRouter == null) return;
    ApplyFormToModel();

    btnFetchLists.IsEnabled = false;
    var client = new Network.MikrotikRestClient(_selectedRouter);

    // Запрашиваем уникальные списки из Mikrotik
    var availableLists = await client.GetAddressListNamesAsync();

    if (availableLists.Count > 0)
    {
        string currentSelection = cmbAddressLists.Text;
        cmbAddressLists.ItemsSource = availableLists;
        cmbAddressLists.Text = currentSelection;
        MessageBox.Show($"Найдено {availableLists.Count} списков файрвола.", "Mikrotik API",
        MessageBoxButton.OK, MessageBoxImage.Information);
    }
    else
    {
        MessageBox.Show("Не удалось получить списки. Проверьте связь или права пользователя.", "Ошибка API", MessageBoxButton.OK, MessageBoxImage.Warning);
    }
    btnFetchLists.IsEnabled = true;
}

private async void BtnTest_Click(object sender, RoutedEventArgs e)
{
    if (_selectedRouter == null) return;
    ApplyFormToModel();

    btnTest.IsEnabled = false;
    lblTestStatus.Text = "Проверка...";
    lblTestStatus.Foreground = Brushes.Orange;

    var client = new Network.MikrotikRestClient(_selectedRouter);
    var result = await client.TestConnectionAsync();

    switch (result)
    {
        case ConnectionStatus.Success:
            lblTestStatus.Text = "Связь установлена!";
            lblTestStatus.Foreground = Brushes.Green;
            break;
        case ConnectionStatus.Unauthorized:
            lblTestStatus.Text = "Ошибка: Нет прав / неверный логин/пароль.";
            lblTestStatus.Foreground = Brushes.Red;
            break;
        case ConnectionStatus.Unreachable:
            lblTestStatus.Text = "Роутер недоступен (Таймаут/IP).";
            lblTestStatus.Foreground = Brushes.Red;
            break;
    }
    btnTest.IsEnabled = true;
}

```

```

private void BtnSave_Click(object sender, RoutedEventArgs e)
{
    ApplyFormToModel();
    if (_routers != null)
    {
        RouterStorage.Save(_routers);
        MessageBox.Show("Конфигурация REST API успешно сохранена!", "Сохранено", MessageBoxButton.OK,
        MessageBoxImage.Information);
        LoadData();
    }
}
}
}
}
}

```

```

---frmSettings.xaml---
<Window x:Class="EDIPSearch.frmSettings"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:local="clr-namespace:EDIPSearch"
mc:Ignorable="d"
Title="Бортовой компьютер: Настройки конфигурации" Height="450" Width="800" Background="#11161B"
Foreground="#E0E6ED"
Icon="pack://application:,,,/app_icon.ico">
<Window.Resources>
<ResourceDictionary>
<ResourceDictionary.MergedDictionaries>
<ResourceDictionary Source="EliteTheme.xaml" />
</ResourceDictionary.MergedDictionaries>
</ResourceDictionary>
</Window.Resources>

```

```

<DockPanel Margin="10">
<!-- НИЖНЯЯ ПАНЕЛЬ С КНОПКАМИ: Развели по краям и отделили оранжевой линией -->
<Border DockPanel.Dock="Bottom" BorderBrush="#33FF7100" BorderThickness="0,1,0,0"
Padding="0,10,0,0" Margin="0,10,0,0">
<Grid>
<Button x:Name="btnCancel" Style="{StaticResource EliteButton}" Content="ОТМЕНА"
HorizontalAlignment="Left" Width="100" Height="28"/>
<Button x:Name="btnOK" Style="{StaticResource EliteButton}" Content="СОХРАНИТЬ"
HorizontalAlignment="Right" Width="100" Height="28"/>
</Grid>
</Border>

```

```

<!-- ОСНОВНОЙ БЛОК ВКЛАДОК -->
<TabControl>
<TabItem Header="Основные настройки">
<!-- Padding="10" на самом гриде создаст одинаковые идеальные отступы от краев вкладки со всех
сторон -->
<Grid Margin="0,10,0,0">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<!-- Строка 1: Путь к ED -->
<RowDefinition Height="Auto"/>
<!-- Строка 2: Хранилище -->
<RowDefinition Height="Auto"/>
<!-- Строка 3: Чекбокс отслеживания -->
</Grid.RowDefinitions>
<Grid.ColumnDefinitions>
<!-- Фиксированная ширина для подписей, чтобы левый край полей ввода был идеально ровным -->
<ColumnDefinition Width="230"/>
<!-- Поля ввода (*) заберут абсолютно всё свободное место между текстом и кнопкой -->
<ColumnDefinition Width="*"/>
<!-- Кнопка-иконка четко зафиксирована на правом краю -->
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>

```

```

<!-- СТРОКА 1: ПУТЬ К ELITE DANGEROUS -->
<Label Content="Путь к каталогу Elite Dangerous:"
Grid.Row="0" Grid.Column="0"
VerticalAlignment="Center" Margin="0,8"/>

```

```

<TextBox x:Name="tbEdPath"
Grid.Row="0" Grid.Column="1"
Height="26" Margin="5,8,10,8"
VerticalContentAlignment="Center"/>

<Button x:Name="btnEDOpen"
Grid.Row="0" Grid.Column="2"
Style="{StaticResource EliteButton}" Content="□ "
Width="32" Height="26" Margin="0,8" Padding="0" FontSize="14"
ToolTip="Выбрать каталог игры"/>

<!-- СТРОКА 2: ХРАНИЛИЩЕ ДАННЫХ ПРИЛОЖЕНИЯ -->
<Label Content="Хранилище данных приложения:"
Grid.Row="1" Grid.Column="0"
VerticalAlignment="Center" Margin="0,8"/>

<!-- Теперь это поле имеет точно такую же длину и выравнивание, как верхнее -->
<TextBox x:Name="tbWorkingDir"
Grid.Row="1" Grid.Column="1"
Height="26" Margin="5,8,10,8"
VerticalContentAlignment="Center"/>

<!-- Кнопка встала строго под верхней, образуя ровную линию правого края -->
<Button x:Name="btnWorkingDir"
Grid.Row="1" Grid.Column="2"
Style="{StaticResource EliteButton}" Content="□ "
Width="32" Height="26" Margin="0,8" Padding="0" FontSize="14"
ToolTip="Выбрать папку хранилища"/>

<!-- СТРОКА 3: ЧЕКБОКС АВТО-ОТСЛЕЖИВАНИЯ -->
<!-- Размещаем его во 2-й колонке (под началом текстовых полей), растягивая на 2 колонки вправо -->
<CheckBox x:Name="cbListenLog"
Grid.Row="2" Grid.Column="1" Grid.ColumnSpan="2"
Content="Отслеживать активный лог-файл в реальном времени"
Margin="5,15,0,0"/>
</Grid>
</TabItem>

<!-- ВКЛАДКА 2: ФИЛЬТРЫ АДРЕСОВ (ListView превращен в таблицу) -->
<TabItem Header="Фильтры" ToolTip="Пул исключений IP-адресов">
<DockPanel Margin="5">
<!-- Тулбар для добавления -->
<ToolBar DockPanel.Dock="Top" Margin="0,0,0,10">
<Label Content="Добавить IP/Подсеть:" VerticalAlignment="Center" Foreground="#FF7100"
Margin="0,0,5,0"/>
<TextBox x:Name="tbAddress" Width="180" Height="24" VerticalContentAlignment="Center"/>

<!-- Плюс и Минус теперь ровные центрированные кнопки -->
<Button Style="{StaticResource EliteButton}" x:Name="tbFilterAdd" Width="26" Height="24"
Margin="6,0,0,0" Padding="0" FontWeight="Bold" Content="+"/>
<Button Style="{StaticResource EliteButton}" x:Name="tbFilterRemove" Width="26" Height="24"
Margin="4,0,0,0" Padding="0" FontWeight="Bold" Content="-"/>
</ToolBar>

<!-- ТАБЛИЦА ФИЛЬТРОВ: Отредерена через GridView колонок -->
<ListView x:Name="lvFilters" Background="Transparent" BorderThickness="0" Foreground="#E0E6ED">
<!-- Умный стиль строк (остается без изменений, твоя раскраска комментариев) -->
<ListView.ItemContainerStyle>
<Style TargetType="ListViewItem" BasedOn="{StaticResource {x:Type ListViewItem}}">
<Style.Triggers>
<DataTrigger Binding="{Binding IsComment}" Value="True">
<Setter Property="Foreground" Value="#55FF7100"/>
<Setter Property="FontStyle" Value="Italic"/>
<Setter Property="Focusable" Value="False"/>
</DataTrigger>
</Style.Triggers>
</Style>
</ListView.ItemContainerStyle>

```

```

<ListView.View>
<GridView>
<GridView.ColumnHeaderContainerStyle>
<Style TargetType="GridViewColumnHeader">
<Setter Property="Background" Value="#1B2228"/>
<Setter Property="Foreground" Value="#FF7100"/>
<Setter Property="FontWeight" Value="Bold"/>
<Setter Property="BorderBrush" Value="#FF7100"/>
<Setter Property="BorderThickness" Value="0,0,0,1"/>
<Setter Property="Padding" Value="12,6"/>
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="GridViewColumnHeader">
<Border x:Name="HeaderBorder" Background="{TemplateBinding Background}"
BorderBrush="{TemplateBinding BorderBrush}" BorderThickness="{TemplateBinding BorderThickness}"
Padding="{TemplateBinding Padding}">
<!-- ИСПРАВЛЕНО: Центрируем заголовки для Маски и Размера, если это указано в колонке -->
<ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center"/>
</Border>
<ControlTemplate.Triggers>
<Trigger Property="IsMouseOver" Value="True">
<Setter TargetName="HeaderBorder" Property="Background" Value="#25FF7100"/>
<Setter Property="Foreground" Value="#FFA000"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>
</GridView.ColumnHeaderContainerStyle>

<!-- Колонки таблицы -->
<!-- 1. IP адрес (выравнивание по левому краю) -->
<GridViewColumn Header="IP АДРЕС / ДИАПАЗОН ИСКЛЮЧЕНИЯ" DisplayMemberBinding="{Binding IP}"
Width="430"/>

<!-- 2. Маска (убрали ошибочный параметр) -->
<GridViewColumn Header="МАСКА" Width="140">
<GridViewColumn.CellTemplate>
<DataTemplate>
<TextBlock Text="{Binding Mask}" TextAlignment="Center" Width="130" Foreground="{Binding
RelativeSource={RelativeSource AncestorType=ListViewItem}, Path=Foreground}"/>
</DataTemplate>
</GridViewColumn.CellTemplate>
</GridViewColumn>

<!-- 3. Размер пула (убрали ошибочный параметр) -->
<GridViewColumn Header="РАЗМЕР ПУЛА" Width="140">
<GridViewColumn.CellTemplate>
<DataTemplate>
<TextBlock Text="{Binding Size}" TextAlignment="Center" Width="130" Foreground="{Binding
RelativeSource={RelativeSource AncestorType=ListViewItem}, Path=Foreground}"/>
</DataTemplate>
</GridViewColumn.CellTemplate>
</GridViewColumn>
</GridView>
</ListView.View>
</ListView>
</DockPanel>
</TabItem>

<!-- ВКЛАДКА 3: ИНТЕГРАЦИЯ -->
<TabItem Header="Интеграция" ToolTip="Связь с маршрутизатором">
<Grid Margin="15">
<TextBlock Text="Настройки интеграции REST API управляются через отдельный модуль на главном
экране."
Foreground="#88E0E6ED" HorizontalAlignment="Center" VerticalAlignment="Center" TextWrapping="Wrap"
TextAlignment="Center"/>
</Grid>
</TabItem>
</TabControl>
</DockPanel>
</Window>

```

```

---frmSettings.xaml.cs---
using ipinpool; // Твое пространство имен для wiseIPList и IPclass
using System;
using System;
using System.Collections.Generic;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;
using Microsoft.Win32;

namespace EDIPSearch;

public partial class frmSettings : Window
{
    // Список строк для отображения в ListView (содержит и IP, и комментарии)
    private List<FilterRowItem> _uiRows = new List<FilterRowItem>();

    // Твой оригинальный класс-фильтр, который мы наполним для передачи в главное окно
    private wiseIPList _filterListForEngine = new wiseIPList();

    public frmSettings()
    {
        InitializeComponent();

        btnOK.Click += BtnOK_Click;
        btnCancel.Click += BtnCancel_Click;
        tbFilterAdd.Click += TbFilterAdd_Click;
        tbFilterRemove.Click += TbFilterRemove_Click;

        this.Loaded += FrmSettings_Loaded;
    }

    private void FrmSettings_Loaded(object sender, RoutedEventArgs e)
    {
        // Просто считываем уже гарантированно инициализированные параметры
        tbEdPath.Text = Properties.Settings.Default.EDLogFolder;

        string defaultDataDir =
            System.IO.Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments),
                "EDIPSearch");
        tbWorkingDir.Text = string.IsNullOrEmpty(Properties.Settings.Default.DataFolder) ? defaultDataDir :
            Properties.Settings.Default.DataFolder;

        cbListenLog.IsChecked = Properties.Settings.Default.AutoMonitoringEnabled;

        // Подвязываем клики по кнопкам-иконкам папок
        btnEDOpen.Click += BtnEDOpen_Click;
        btnWorkingDir.Click += BtnWorkingDir_Click;

        LoadFiltersFile();
    }

    private void LoadFiltersFile()
    {
        _uiRows.Clear();
        string filePath = System.IO.Path.Combine(tbWorkingDir.Text, "filters.txt");

```

```

if (!File.Exists(filePath)) return;

try
{
    string[] lines = File.ReadAllLines(filePath);
    foreach (string line in lines)
    {
        string trimmed = line.Trim();
        if (string.IsNullOrEmpty(trimmed)) continue;

        // Сценарий 2: Чистый комментарий (начинается с #)
        if (trimmed.StartsWith("#"))
        {
            _uiRows.Add(new FilterRowItem { IsComment = true, RawText = line });
            continue;
        }

        // Сценарий 3 при загрузке: В строке есть и IP, и комментарий через #
        if (trimmed.Contains("#"))
        {
            // Бьем строго по первому символу #
            int hashIndex = line.IndexOf('#');
            string ipPart = line.Substring(0, hashIndex).Trim();
            string commentPart = line.Substring(hashIndex).Trim(); // Сохраняет сам символ # и текст за ним

            // Валидируем IP-часть (авто-допишет /32 если надо и проверит дубли)
            if (ValidateAndNormalizeIp(ipPart, out string normalizedIp, out IPclass? ipObj))
            {
                // Разворачиваем в наборе данных: сначала добавляем комментарий, затем IP
                _uiRows.Add(new FilterRowItem { IsComment = true, RawText = commentPart });
                _uiRows.Add(new FilterRowItem { IsComment = false, RawText = normalizedIp, IpObject = ipObj });
            }
            continue;
        }

        // Сценарий 1 при загрузке: Чистый IP/маска
        if (ValidateAndNormalizeIp(trimmed, out string cleanIp, out IPclass? pureIpObj))
        {
            _uiRows.Add(new FilterRowItem { IsComment = false, RawText = cleanIp, IpObject = pureIpObj });
        }
    }

    RefreshGrid();
}
catch (Exception ex)
{
    MessageBox.Show($"Ошибка чтения: {ex.Message}");
}

private void RefreshGrid()
{
    lvFilters.ItemsSource = null;
    lvFilters.ItemsSource = _uiRows;
}

private void TbFilterAdd_Click(object sender, RoutedEventArgs e)
{
    string input = tbAddress.Text.Trim();
    if (string.IsNullOrEmpty(input)) return;

    // SCENARIO 2: Пользователь добавляет чистый #комментарий
    if (input.StartsWith("#"))
    {
        var commentRow = new FilterRowItem { IsComment = true, RawText = tbAddress.Text }; // Сохраняем с пробелами как ввёл
        _uiRows.Add(commentRow);
        RefreshGrid();
        lvFilters.ScrollIntoView(commentRow);
    }
}

```

```

tbAddress.Clear();
return;
}

// SCENARIO 3: Пользователь добавляет комбинированную строку "ip[/mask] #комментарий"
if (input.Contains("#"))
{
    int hashIndex = tbAddress.Text.IndexOf('#');
    string ipPart = tbAddress.Text.Substring(0, hashIndex).Trim();
    string commentPart = tbAddress.Text.Substring(hashIndex).Trim(); // Забирает # и всё что после

    // Проводим валидацию секции IP
    if (ValidateAndNormalizeIp(ipPart, out string normalizedIp, out IPclass? ipObj))
    {
        // Если валидация прошла – последовательно добавляем две строки
        var commentRow = new FilterRowItem { IsComment = true, RawText = commentPart };
        var ipRow = new FilterRowItem { IsComment = false, RawText = normalizedIp, IpObject = ipObj };

        _uiRows.Add(commentRow);
        _uiRows.Add(ipRow);

        RefreshGrid();
        lvFilters.ScrollIntoView(ipRow);
        tbAddress.Clear();
    }
    else
    {
        MessageBox.Show("Неверный формат IP-адреса, либо такой адрес уже существует в таблице!", "Ошибка валидации",
            MessageBoxButton.OK, MessageBoxImage.Warning);
    }
    return;
}

// SCENARIO 1: Пользователь добавляет чистый ip[/mask]
if (ValidateAndNormalizeIp(input, out string singleNormalizedIp, out IPclass? singleIpObj))
{
    var ipRow = new FilterRowItem { IsComment = false, RawText = singleNormalizedIp, IpObject =
        singleIpObj };
    _uiRows.Add(ipRow);
    RefreshGrid();
    lvFilters.ScrollIntoView(ipRow);
    tbAddress.Clear();
}
else
{
    MessageBox.Show("Неверный формат IP-адреса, либо такой адрес уже существует!", "Ошибка валидации",
        MessageBoxButton.OK, MessageBoxImage.Warning);
}

/// <summary>
/// Кнопка МИНУС (-): Удаление строки (хоть IP, хоть комментария)
/// </summary>
private void TbFilterRemove_Click(object sender, RoutedEventArgs e)
{
    if (lvFilters.SelectedItem is FilterRowItem selectedItem)
    {
        _uiRows.Remove(selectedItem);
        RefreshGrid();
    }
}

private void BtnOK_Click(object sender, RoutedEventArgs e)
{
    try
    {
        string targetFolder = tbWorkingDir.Text.Trim();
        if (!Directory.Exists(targetFolder)) Directory.CreateDirectory(targetFolder);

        string filePath = System.IO.Path.Combine(targetFolder, "filters.txt");

```

```

// Пересохраняем файл, сохраняя ВСЮ структуру и комментарии на своих местах!
using (StreamWriter writer = new StreamWriter(filePath, false, System.Text.Encoding.UTF8))
{
    foreach (var row in _uiRows)
    {
        if (row.IsComment)
        {
            writer.WriteLine(row.RawText); // Пишем комментарий как есть
        }
        else
        {
            writer.WriteLine(row.IpObject?.ToString() ?? row.RawText); // Пишем IP
        }
    }
}

// Параллельно собираем чистый wiseIPList для отправки в ядро программы (без комментариев)
_filterListForEngine = new wiseIPList();
foreach (var row in _uiRows)
{
    if (!row.IsComment && row.IpObject != null)
    {
        _filterListForEngine.AddAddress(row.IpObject);
    }
}

// Сохраняем системные настройки
Properties.Settings.Default.EDLogFolder = tbEdPath.Text.Trim();
Properties.Settings.Default.DataFolder = targetFolder;
Properties.Settings.Default.AutoMonitoringEnabled = cbListenLog.IsChecked ?? false;
Properties.Settings.Default.Save();

// Здесь передай _filterListForEngine в свое главное окно, если нужно обновить фильтр на лету

this.DialogResult = true;
this.Close();
}
catch (Exception ex) { MessageBox.Show($"Ошибка: {ex.Message}"); }
}

private void BtnCancel_Click(object sender, RoutedEventArgs e)
{
    this.DialogResult = false;
    this.Close();
}
/// <summary>
/// Валидация и унификация IP-адреса. Дописывает /32 при отсутствии маски и проверяет на дубликаты.
/// </summary>
/// <param name="rawIpInput">Сырая строка ввода (например, "1.2.3.4" или "10.0.0.0/8")</param>
/// <param name="cleanNormalizedIp">Выходная унифицированная строка</param>
/// <param name="parsedObj">Выходной готовый объект IPclass</param>
private bool ValidateAndNormalizeIp(string rawIpInput, out string cleanNormalizedIp, out IPclass? parsedObj)
{
    cleanNormalizedIp = rawIpInput.Trim();
    parsedObj = null;

    if (string.IsNullOrEmpty(cleanNormalizedIp)) return false;

    // Если пользователь забыл указать маску, принудительно дописываем /32 для унификации
    if (!cleanNormalizedIp.Contains("/"))
    {
        cleanNormalizedIp += "/32";
    }

    // Пробуем распарсить твоим оригинальным методом
    parsedObj = IPclass.Parse(cleanNormalizedIp);
    if (parsedObj == null) return false;
}

```

```

// Особого смысла проверять всю математику пула нет, проверяем физическое совпадение IP-строки в
таблице
foreach (var row in _uiRows)
{
    if (!row.IsComment && row.IpObject != null)
    {
        // Сравниваем чистые IP (например, "194.87.147.176/32")
        if (row.IpObject.ToString().Equals(parsedObj.ToString(), StringComparison.OrdinalIgnoreCase))
        {
            return false; // Такой адрес уже физически есть в списке
        }
    }
}

return true;
}

/// <summary>
/// Продвинутый автоматический поиск папки журналов Elite Dangerous
/// </summary>
private string AutoDetectEliteDangerousPath()
{
    // 1. Стандартный путь Windows (Saved Games), где игра хранит логи у 95% пилотов
    string userProfile = Environment.GetFolderPath(Environment.SpecialFolder.UserProfile);
    string defaultLogPath = System.IO.Path.Combine(userProfile, "Saved Games", "Frontier Developments",
    "Elite Dangerous");

    if (Directory.Exists(defaultLogPath))
    {
        return defaultLogPath;
    }

    // 2. Подстраховка: Пробуем найти через реестр Steam, если папка кастомная
    try
    {
        using (var key = Registry.CurrentUser.OpenSubKey(@"Software\Valve\Steam"))
        {
            if (key != null)
            {
                string? steamPath = key.GetValue("SteamPath") as string;
                if (!string.IsNullOrEmpty(steamPath))
                {
                    // Проверяем дефолтную библиотеку стима
                    string steamLogPath = System.IO.Path.Combine(steamPath, "steamapps", "common", "Elite Dangerous");
                    // Сами журналы лежат в профиле пользователя, но если пользователь ищет корневой каталог:
                    if (Directory.Exists(steamLogPath)) return steamLogPath;
                }
            }
        }
    }
    catch { }

    return string.Empty; // Если определить не удалось, оставляем пустым для ручного ввода
}

/// <summary>
/// Выбор папки логов Elite Dangerous
/// </summary>
private void BtnEDOpen_Click(object sender, RoutedEventArgs e)
{
    var dialog = new OpenFolderDialog
    {
        Title = "Укажите папку с журналами (Journal*.log) Elite Dangerous",
        InitialDirectory = Directory.Exists(tbEdPath.Text) ? tbEdPath.Text :
        Environment.GetFolderPath(Environment.SpecialFolder.UserProfile)
    };

    if (dialog.ShowDialog() == true)
    {
        tbEdPath.Text = dialog.FolderName;
    }
}

```

```

/// <summary>
/// Выбор папки хранилища данных приложения
/// </summary>
private void BtnWorkingDir_Click(object sender, RoutedEventArgs e)
{
    var dialog = new OpenFolderDialog
    {
        Title = "Выберите папку для хранения данных EDIPSearch (filters.txt)",
        InitialDirectory = Directory.Exists(tbWorkingDir.Text) ? tbWorkingDir.Text :
        Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments)
    };

    if (dialog.ShowDialog() == true)
    {
        tbWorkingDir.Text = dialog.FolderName;
    }
}

}

public class FilterRowItem
{
    public bool IsComment { get; set; }
    public string RawText { get; set; } = string.Empty;

    public string IP => IsComment ? RawText : IpObject?.IP ?? string.Empty;
    public string Mask => IsComment ? string.Empty : (IpObject?.PoolSize.ToString() ?? string.Empty);
    public string Size => IsComment ? string.Empty : CalculateSize(IpObject?.PoolSize ?? 32);

    public IPclass? IpObject { get; set; }

    private string CalculateSize(int poolSize)
    {
        long usableAddresses = 0;

        if (poolSize == 32) usableAddresses = 1;
        else if (poolSize == 31) usableAddresses = 2;
        else
        {
            long total = (long)Math.Pow(2, 32 - poolSize);
            usableAddresses = total - 2;
        }

        // ИСПРАВЛЕНО: Жестко задаем пробел в качестве разделителя тысяч
        var nfi = new System.Globalization.NumberFormatInfo
        {
            NumberGroupSeparator = " ", // Разделитель - пробел
            NumberDecimalDigits = 0 // Отключаем копейки/дробную часть
        };

        // Форматируем число с использованием нашей маски
        return usableAddresses.ToString("N", nfi);
    }
}

---MainWindow.xaml---
<Window x:Class="EDIPSearch.MainWindow"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:local="clr-namespace:EDIPSearch"
mc:Ignorable="d"
Title="Elite Dangerous: Log Parser & Router Sync [CMDR Edition]" Height="450" Width="800"
Background="#11161B" Foreground="#E0E6ED"
Icon="pack://application:,,,/app_icon.ico">

```

```

<Window.Resources>
<ResourceDictionary>
<ResourceDictionary.MergedDictionaries>
<!-- Подключаем созданный файл темы из ресурсов проекта -->
<ResourceDictionary Source="EliteTheme.xaml" />
</ResourceDictionary.MergedDictionaries>
</ResourceDictionary>
</Window.Resources>

<Grid Margin="15">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<!-- Панель управления -->
<RowDefinition Height="*/>
<!-- Основной контент -->
<RowDefinition Height="Auto"/>
<!-- Статус бар -->
</Grid.RowDefinitions>

<!-- ВЕРХНЯЯ ПАНЕЛЬ: Твои старые кнопки + новые переключатели -->
<Border Grid.Row="0" Background="#1B2228" CornerRadius="4" Padding="10" Margin="0,0,0,15"
BorderBrush="#33FF7100" BorderThickness="1">
<Grid>
<!-- Используем WrapPanel, чтобы уместить И ТВОИ СТАРЫЕ КНОПКИ, и новые элементы -->
<WrapPanel Orientation="Horizontal" VerticalAlignment="Center">

<!-- КНОПКИ ИЗ ТВОЕЙ СТАРОЙ ПРОГРАММЫ (Верни их сюда со всеми их именами и событиями Click) -->
<Button x:Name="btnOldParse" Content="АНАЛИЗ ЛОГОВ" Margin="0,0,10,0" Click="BtnOldParse_Click"
Style="{StaticResource EliteButton}"/>
<Button Content="НАСТРОЙКИ" Margin="0,0,20,0" Click="BtnOldSettings_Click" Style="{StaticResource
EliteButton}" />

<!-- Новые элементы управления Mikrotik REST API -->
<Button Content="□ MIKROTIK REST" Style="{StaticResource EliteButton}" Margin="0,0,20,0"
Click="BtnOpenSettings_Click"/>

<CheckBox x:Name="chkEnableMonitoring" Content="АВТО-ОТСЛЕЖИВАНИЕ"
Foreground="#FF7100" FontWeight="Bold" Margin="0,0,20,0" VerticalAlignment="Center"
Checked="ChkEnableMonitoring_Checked" Unchecked="ChkEnableMonitoring_Unchecked" IsChecked="True"/>

<!-- Статус игры -->
<StackPanel Orientation="Horizontal" Margin="20,0,0,0">
<TextBlock x:Name="txtGameIcon" Text="□" FontSize="16" VerticalAlignment="Center"
Margin="0,0,5,0"/>
<TextBlock x:Name="txtGameStatus" Text="ИГРА НЕ ЗАПУЩЕНА" Foreground="#FF4500" FontWeight="Bold"
VerticalAlignment="Center"/>
</StackPanel>
</Panel>
</WrapPanel>
</Grid>
</Border>

<!-- ЦЕНТРАЛЬНАЯ ЧАСТЬ: Разделена на две колонки -->
<Grid Grid.Row="1">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="2*"/>
<!-- Основная таблица адресов (твоя существующая) -->
<ColumnDefinition Width="1*"/>
<!-- Новая панель: Информативность / Живой лог -->
</Grid.ColumnDefinitions>

<!-- Твоя существующая таблица адресов (пример структуры, привяжи свою) -->
<Border Grid.Column="0" Background="#1B2228" CornerRadius="4" BorderBrush="#22FF7100"
BorderThickness="1" Margin="0,0,10,0" Padding="5">
<Grid>
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="*/>
</Grid.RowDefinitions>
<!-- Счетчик адресов прямо над таблицей -->
<StackPanel Orientation="Horizontal" Margin="5">
<TextBlock Text="ВСЕГО АДРЕСОВ В ТАБЛИЦЕ: " Foreground="#E0E6ED" FontWeight="Bold"/>
<TextBlock x:Name="txtTotalAddressesCount" Text="0" Foreground="#FFA000" FontWeight="Bold"/>

```

```
</StackPanel>
```

```
<DataGrid x:Name="dgAddresses" Grid.Row="1"
Background="Transparent"
RowBackground="#161C20"
AlternatingRowBackground="#1B2228"
Foreground="#E0E6ED"
BorderThickness="0"
HeadersVisibility="Column"
GridLinesVisibility="Horizontal"
HorizontalGridLinesBrush="#1A33FF71"
AutoGenerateColumns="False"
CanUserAddRows="False"
SelectionUnit="FullRow"
IsReadOnly="True"
ScrollViewer.HorizontalScrollBarVisibility="Disabled">
<!-- Отключаем горизонтальный скролл, чтобы убрать белый артефакт в углу -->
```

```
<DataGrid.Resources>
```

```
<!-- 1. СТИЛИЗАЦИЯ ЧЕКБОКСА В ТАБЛИЦЕ (Делаем его оранжевым под стиль Элиты) -->
<Style TargetType="CheckBox">
<Setter Property="Background" Value="#11161B"/>
<Setter Property="BorderBrush" Value="#FF7100"/>
<Setter Property="Foreground" Value="#E0E6ED"/>
</Style>
```

```
<!-- 2. СТИЛИЗАЦИЯ ПОЛЗУНКА СКРОЛЛБАРА -->
<Style x:Key="ScrollThumbs" TargetType="Thumb">
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="Thumb">
<!-- Рисуем аккуратную оранжевую полосу скругленной формы -->
<Grid x:Name="Grid">
<Border HorizontalAlignment="Stretch" VerticalAlignment="Stretch"
Background="#55FF7100" CornerRadius="3" MinHeight="20" />
</Grid>
<ControlTemplate.Triggers>
<!-- При наведении мыши ползунок загорается ярким неоном -->
<Trigger Property="IsMouseOver" Value="true">
<Setter Property="Background" Value="#FF7100"/>
</Trigger>
</ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>
```

```
<!-- 3. ГЛОБАЛЬНЫЙ ШАБЛОН ДЛЯ ВСЕХ СКРОЛЛБАРОВ ТАБЛИЦЫ -->
<Style TargetType="ScrollBar">
<Setter Property="Background" Value="#11161B" />
<Setter Property="Width" Value="8" />
<Setter Property="Template">
<Setter.Value>
<ControlTemplate TargetType="ScrollBar">
<Grid x:Name="Bg" Background="#11161B" SnapsToDevicePixels="true">
<!-- Трек, по которому бежит ползунок -->
<Track x:Name="PART_Track" IsDirectionReversed="true" IsEnabled="{TemplateBinding IsEnabled}">
<Track.DecreaseRepeatButton>
<!-- Пустая прозрачная кнопка вместо верхней стрелки Windows -->
<RepeatButton Command="ScrollBar.LineUpCommand" Opacity="0" Focusable="false" />
</Track.DecreaseRepeatButton>
<Track.Thumb>
<!-- Наш кастомный оранжевый ползунок -->
<Thumb Style="{StaticResource ScrollThumbs}" />
</Track.Thumb>
<Track.IncreaseRepeatButton>
<!-- Пустая прозрачная кнопка вместо нижней стрелки Windows -->
<RepeatButton Command="ScrollBar.LineDownCommand" Opacity="0" Focusable="false" />
</Track.IncreaseRepeatButton>
</Track>
</Grid>
</ControlTemplate>
</Setter.Value>
```

```

</Setter>
</Style>
</DataGrid.Resources>
<DataGrid.ContextMenu>
<!-- Убрали ошибочный AllowsTransparency. Оставляем только базовые свойства -->
<ContextMenu Background="#1B2228" BorderBrush="#FF7100" BorderThickness="1">
<MenuItem Header="Выгрузить список в TXT..." Click="BtnExportText_Click"/>
</ContextMenu>
</DataGrid.ContextMenu>
<!-- Стилизация заголовков (остается без изменений) -->
<DataGrid.ColumnHeaderStyle>
<Style TargetType="DataGridColumnHeader">
<Setter Property="Background" Value="#1B2228"/>
<Setter Property="Foreground" Value="#FF7100"/>
<Setter Property="FontWeight" Value="Bold"/>
<Setter Property="BorderBrush" Value="#FF7100"/>
<Setter Property="BorderThickness" Value="0,0,0,1"/>
<Setter Property="Padding" Value="10,6"/>
</Style>
</DataGrid.ColumnHeaderStyle>

<!-- Колонки таблицы -->
<DataGrid.Columns>
<DataGridTextColumn Header="IP АДРЕС" Binding="{Binding IP}" Width="2*"/>
<DataGridTextColumn Header="МАСКА" Binding="{Binding Mask}" Width="1*"/>
</DataGrid.Columns>
</DataGrid>

</Grid>
</Border>

<!-- ПРАВАЯ ПАНЕЛЬ: Статистика файла + Лента активности -->
<Border Grid.Column="1" Background="#1B2228" CornerRadius="4" BorderBrush="#33FF7100"
BorderThickness="1" Padding="10">
<Grid>
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<!-- Заголовок -->
<RowDefinition Height="Auto"/>
<!-- Имя файла -->
<RowDefinition Height="Auto"/>
<!-- Счетчик строк -->
<RowDefinition Height="*/>
<!-- Список логов -->
</Grid.RowDefinitions>

<TextBlock Text="МОНИТОРИНГ ЖУРНАЛА:" Foreground="#FF7100" FontWeight="Bold" Margin="0,0,0,8"/>

<!-- ДИАГНОСТИКА: Имя файла лога -->
<StackPanel Grid.Row="1" Orientation="Horizontal" Margin="0,0,0,4">
<TextBlock Text="Файл:" Foreground="#888" FontSize="11"/>
<TextBlock x:Name="txtCurrentLogFile" Text="нет файла" Foreground="#E0E6ED" FontSize="11"
TextTrimming="CharacterEllipsis" MaxWidth="180"/>
</StackPanel>

<!-- ДИАГНОСТИКА: Прочитано строк -->
<StackPanel Grid.Row="2" Orientation="Horizontal" Margin="0,0,0,12">
<TextBlock Text="Прочитано строк: " Foreground="#888" FontSize="11"/>
<TextBlock x:Name="txtTotalLinesRead" Text="0" Foreground="#FFA000" FontSize="11"
FontWeight="Bold"/>
</StackPanel>

<!-- Лента событий -->
<ListBox x:Name="lstLiveActivity" Grid.Row="3" Background="#11161B" Foreground="#FFA000"
BorderThickness="0" FontSize="11">
<ListBox.ItemTemplate>
<DataTemplate>
<TextBlock Text="{Binding}" Margin="2"/>
</DataTemplate>
</ListBox.ItemTemplate>
</ListBox>

```

```

</Grid>
</Border>
</Grid>

<!-- НИЖНИЙ СТАТУС БАР -->
<StatusBar Grid.Row="2" Background="Transparent" Foreground="#888" Margin="0,10,0,0" Padding="0">
<StatusBarItem Padding="0">
<StackPanel Orientation="Horizontal">
<TextBlock x:Name="tbStatusProp" Text="Ожидание..." Foreground="#FFA000"/>
<TextBlock Text=" | " Foreground="#333"/>
<TextBlock x:Name="tbStatusVal" Text="" Foreground="#E0E6ED"/>
</StackPanel>
</StatusBarItem>
</StatusBar>
</Grid>
</Window>

```

```

---MainWindow.xaml.cs---
using EDIPSearch.Core;
using EDIPSearch.Models;
using EDIPSearch.Network;
using EDIPSearch.Properties;
using ipinpool;
using Microsoft.Win32;
using System.IO;
using System.Text;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;
using System.Windows.Threading;
using System.Diagnostics;
using System.Linq;
using System.Timers;
using System.Text.RegularExpressions;

namespace EDIPSearch;

/// <summary>
/// Interaction logic for MainWindow.xaml
/// </summary>
public partial class MainWindow : Window
{
    wiseIPList EDList = new wiseIPList();
    string WorkingDir = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
    string EDLogs = string.Empty;
    RouterList RouterList = new RouterList();

    private MikrotikSyncCore _syncCore = new MikrotikSyncCore();
    private System.Timers.Timer _gameCheckTimer;
    private bool _isEliteRunning = false;
    //private FileSystemWatcher? _logWatcher;

    private System.Timers.Timer? _logReadTimer; // Таймер для чтения строк лога
    private string? _currentLogFilePath;
    private long _lastLogPosition = 0;
    private int _linesReadCounter = 0; // Наш новый счетчик строк
    public MainWindow()
    {
        InitializeComponent();
        WorkingDir = System.IO.Path.Combine(WorkingDir, "EDIPSearch");
        // Восстанавливаем состояние переключателя из настроек при старте
        chkEnableMonitoring.IsChecked = Properties.Settings.Default.AutoMonitoringEnabled;

        if (Settings.Default.DataFolder != string.Empty)
        {

```

```

WorkingDir = Settings.Default.DataFolder;
}
else
{
Settings.Default.DataFolder = WorkingDir;
}

if (!Directory.Exists(WorkingDir)) Directory.CreateDirectory(WorkingDir);
// if (!Directory.Exists(WorkingDir + "\\routers\\profiles")) Directory.CreateDirectory(WorkingDir
+ "\\routers\\profiles");
// if (!Directory.Exists(WorkingDir + "\\keys")) Directory.CreateDirectory(WorkingDir + "\\keys");
// 2. АВТООПРЕДЕЛЕНИЕ ПУТИ К ЛОГАМ ИГРЫ ПРИ ПЕРВОМ СТАРТЕ
string currentLogFolder = Properties.Settings.Default.EDLogFolder;
if (string.IsNullOrEmpty(currentLogFolder))
{
// Если в конфиге пусто – запускаем перенесенный автопоиск
currentLogFolder = AutoDetectEliteDangerousPath();

if (!string.IsNullOrEmpty(currentLogFolder))
{
Properties.Settings.Default.EDLogFolder = currentLogFolder;
Properties.Settings.Default.Save(); // Фиксируем найденный путь в системе
}
}
//Инициализация Mikrotik REST API
if (!File.Exists(WorkingDir + "\\filters.txt"))
{
EDList.SetDefaultFilters(WorkingDir + "\\filters.txt");
}
else
{
EDList.LoadFilters(WorkingDir + "\\filters.txt");
}
// Создаем экземпляр синхронизатора (например, на уровне MainWindow)
EDList.OnParseStart += EDList_OnParseStart;
EDList.OnParseProceed += EDList_OnParseProceed;
EDList.OnParseComplete += EDList_OnParseComplete;
SetupAndSyncMikrotiksAsync(); //Инициализация и синхронизация маршрутизаторов.
// Task.Run(SetupAndSyncMikrotiksAsync);
// Инициализируем таймер проверки игры (5000 миллисекунд = 5 секунд)
_gameCheckTimer = new System.Timers.Timer(5000);
_gameCheckTimer.Elapsed += GameCheckTimer_Elapsed;
_gameCheckTimer.AutoReset = true;
_gameCheckTimer.Start(); // Запускаем постоянный фоновый опрос

//Загрузка роутеров
// SetupTestRouter(); // <--- Заглушка
//Сохранение настроек - последний этап инициализации.
Settings.Default.Save();
}
private void BtnExportText_Click(object sender, RoutedEventArgs e)
{
// Генерируем динамическую временную метку (например, 20260825_201530)
string timestamp = DateTime.Now.ToString("yyyyMMdd_HH:mm:ss");
string defaultFileName = $"ed_filters_{timestamp}.txt";

var saveFileDialog = new SaveFileDialog
{
Title = "Экспорт пула адресов для сторонних маршрутизаторов",
Filter = "Текстовые файлы (*.txt)|*.txt|Все файлы (*.*)|*.*",
FileName = defaultFileName, // ПОДСТАВЛЕНО: ed_дата_время.txt
InitialDirectory = WorkingDir
};

if (saveFileDialog.ShowDialog() == true)
{
try
{
string targetFilePath = saveFileDialog.FileName;

// Выгружаем данные
using (var writer = new StreamWriter(targetFilePath, false, Encoding.UTF8))
{

```

```

writer.WriteLine("#=====");
writer.WriteLine("# СВОДНЫЙ ПУЛ АДРЕСОВ ИСКЛЮЧЕНИЙ ELITE DANGEROUS");
writer.WriteLine($"## Сгенерировано автоматически: {DateTime.Now:dd.MM.yyyy HH:mm:ss}");
writer.WriteLine("# Подходит для ручного импорта в Keenetic, ASUS, TP-Link");
writer.WriteLine("#=====");
writer.WriteLine();

```

```

// Выгружаем строки из твоего мастер-списка EDList
foreach (var ipObj in EDList.ipTable)
{
writer.WriteLine(ipObj.ToString());
}
}

```

```

tbStatusProp.Text = "Экспорт завершен.";
tbStatusVal.Text = $"Сохранен файл: {System.IO.Path.GetFileName(targetFilePath)}";
}
catch (Exception ex)
{
MessageBox.Show($"Не удалось выгрузить список: {ex.Message}", "Ошибка экспорта",
MessageBoxButton.OK, MessageBoxImage.Error);
}
}
}

```

```

private void GameCheckTimer_Elapsed(object? sender, ElapsedEventArgs e)
{
// Ищем процесс Elite Dangerous в системе
bool currentStatus = Process.GetProcessesByName("EliteDangerous64").Any();

```

```

// Проверяем, изменился ли статус с момента последней проверки
if (currentStatus != _isEliteRunning)
{
_isEliteRunning = currentStatus;

```

```

// Так как таймер работает в фоновом потоке,
// для обновления элементов UI WPF мы обязаны использовать Dispatcher
Dispatcher.Invoke(() =>
{
if (_isEliteRunning)
{
// 2. Иконка и статус для запущенной игры
txtGameIcon.Text = "□"; // Меняем символ-иконку на ракету
txtGameIcon.Foreground = System.Windows.Media.Brushes.Green;
txtGameStatus.Text = "ИГРА ЗАПУЩЕНА";
txtGameStatus.Foreground = System.Windows.Media.Brushes.Green;

```

```

// Запускаем мониторинг логов ТОЛЬКО если пользователь включил галочку в UI
if (chkEnableMonitoring.IsChecked == true)
{
StartLiveLogMonitoring();
}
}
else
{
// Иконка и статус для закрытой игры
txtGameIcon.Text = "□"; // Меняем символ-иконку на стоп-сигнал
txtGameIcon.Foreground = System.Windows.Media.Brushes.Red;
txtGameStatus.Text = "ИГРА НЕ ЗАПУЩЕНА";
txtGameStatus.Foreground = System.Windows.Media.Brushes.Red;

StopLiveLogMonitoring();
}
});

```

```

}
}
private void StartLiveLogMonitoring()
{
try

```

```

{
string logFolder = Properties.Settings.Default.EDLogFolder;
if (!Directory.Exists(logFolder))
{
System.Diagnostics.Debug.WriteLine($"[Watcher] Папка логов не существует: {logFolder}");
return;
}

// 1. Инициализируем или сбрасываем счетчики
_linesReadCounter = 0;
txtTotalLinesRead.Text = "0";

// 2. Принудительно находим самый свежий файл прямо сейчас
UpdateActiveLogFile(logFolder);

// 3. Запускаем высокоточный секундный таймер для чтения дозаписи в файл
if (_logReadTimer == null)
{
_logReadTimer = new System.Timers.Timer(1000); // Опрос раз в 1 секунду
_logReadTimer.Elapsed += LogReadTimer_Elapsed;
_logReadTimer.AutoReset = true;
}
_logReadTimer.Start();

System.Diagnostics.Debug.WriteLine("[Watcher] Таймер мониторинга логов запущен.");
}
catch (Exception ex)
{
System.Diagnostics.Debug.WriteLine($"[Watcher Error] {ex.Message}");
}
}

private void StopLiveLogMonitoring()
{
if (_logReadTimer != null)
{
_logReadTimer.Stop();
_logReadTimer.Dispose();
_logReadTimer = null;
}

_currentLogFilePath = null;
_lastLogPosition = 0;

Dispatcher.Invoke(() =>
{
txtCurrentLogFile.Text = "отключен";
});
System.Diagnostics.Debug.WriteLine("[Watcher] Таймер мониторинга логов остановлен.");
}

/// <summary>
/// Сканирует папку и находит самый свежий лог-файл игры
/// </summary>
private void UpdateActiveLogFile(string logFolder)
{
try
{
if (!Directory.Exists(logFolder))
{
Dispatcher.Invoke(() => txtCurrentLogFile.Text = "Ошб: папка не найдена");
return;
}

var directoryInfo = new DirectoryInfo(logFolder);

// Берем файлы Journal.*.log или просто *.log, отсортированные по времени изменения
var freshLogFile = directoryInfo.GetFiles("Journal.*.log")
.OrderByDescending(f => f.LastWriteTime)
.FirstOrDefault();

```

```

// Если по строгой маске Элиты ничего нет, попробуем поискать любые файлы .log для страховки
if (freshLogFile == null)
{
    freshLogFile = directoryInfo.GetFiles("*.log")
        .OrderByDescending(f => f.LastWriteTime)
        .FirstOrDefault();
}

if (freshLogFile != null)
{
    if (_currentLogFilePath != freshLogFile.FullName)
    {
        _currentLogFilePath = freshLogFile.FullName;

        // ИЗМЕНЕНИЕ ДЛЯ ОТЛАДКИ: Ставим 0 вместо freshLogFile.Length.
        // Это заставит программу при старте сразу вычитать ВСЕ строки из текущего лога игры,
        // счетчик строк мгновенно оживет (покажет 500, 1000 и т.д.), и таблица наполнится.
        _lastLogPosition = 0;

        Dispatcher.Invoke(() =>
        {
            txtCurrentLogFile.Text = freshLogFile.Name;
            txtCurrentLogFile.ToolTip = freshLogFile.FullName;
        });
    }
    else
    {
        Dispatcher.Invoke(() => txtCurrentLogFile.Text = "Логи не найдены в папке");
    }
}
catch (Exception ex)
{
    Dispatcher.Invoke(() => txtCurrentLogFile.Text = $"Ошибка: {ex.Message}");
}

private void LogReadTimer_Elapsed(object? sender, System.Timers.ElapsedEventArgs e)
{
    if (string.IsNullOrEmpty(_currentLogFilePath)) return;

    try
    {
        string logFolder = Properties.Settings.Default.EDLogFolder;
        UpdateActiveLogFile(logFolder);

        var fileInfo = new FileInfo(_currentLogFilePath);
        if (fileInfo.Length == _lastLogPosition) return;
        if (fileInfo.Length < _lastLogPosition) _lastLogPosition = 0;

        // Список для накопления IP, найденных ЗА ОДНУ СЕКУНДУ опроса таймера
        var discoveredIps = new List<IPclass>();
        int linesReadInThisTick = 0;

        using (var stream = new FileStream(_currentLogFilePath, FileMode.Open, FileAccess.Read,
            FileShare.ReadWrite))
        {
            stream.Position = _lastLogPosition;

            using (var reader = new StreamReader(stream, Encoding.UTF8))
            {
                // БЫЛО:
                // string? line;
                // while ((line = reader.ReadLine()) != null) { ... Regex.Match(line, ...) ... }

                // СТАЛО (Защита от оптимизатора Release):

```

```

while (true)
{
string? currentLine = reader.ReadLine();
if (currentLine == null) break; // Выходим, если лог кончился

linesReadInThisTick++;

// Создаем абсолютно изолированную копию строки для потока регулярки.
// Это на 100% запретит JIT-компилятору оптимизировать и перетирать память!
string internalLineCopy = string.Concat(currentLine);
Match match = Regex.Match(internalLineCopy, @"^b(?:[0-9]{1,3}\.){3}[0-9]{1,3}\b");
if (match.Success)
{
IPClass? parsedIp = IPClass.Parse($"{match.Value}/32");
if (parsedIp != null)
{
discoveredIps.Add(parsedIp);
}
}
}

_lastLogPosition = stream.Position;
}
}

// Если мы что-то прочитали или нашли новые IP — отправляем это в UI ОДНИМ СИНХРОННЫМ БЛОКОМ
if (linesReadInThisTick > 0)
{
// Используем Invoke вместо BeginInvoke, чтобы поток таймера подождал, пока WPF гарантированно
перерисует интерфейс
Dispatcher.Invoke(() =>
{
// 1. Обновляем счетчик прочитанных строк лога
_linesReadCounter += linesReadInThisTick;
txtTotalLinesRead.Text = _linesReadCounter.ToString();

// 2. Замораживаем обновление DataGridView на время массового добавления
// (Это предотвратит ложные срабатывания CollectionChanged в WPF)
// Отключаем событие, чтобы UI не штормило, если у тебя там была подписка

foreach (var ip in discoveredIps)
{
// Твой метод проверяет дубликаты и добавляет в ipTable
EDList.AddAddress(ip);
}

// 3. Если были добавлены реально новые IP, принудительно и безопасно обновляем таблицу
// ... Твой код внутри Dispatcher.Invoke в методе LogReadTimer_Elapsed ...
if (discoveredIps.Count > 0)
{
dgAddresses.ItemsSource = null;
dgAddresses.ItemsSource = EDList.ipTable;
txtTotalAddressesCount.Text = EDList.ipTable.Count.ToString();

// АВТОПРОКРУТКА: Проверяем, есть ли элементы в таблице
if (EDList.ipTable.Count > 0)
{
// Забираем самый последний добавленный объект IPClass из таблицы
var lastItem = EDList.ipTable[EDList.ipTable.Count - 1];

// Принудительно заставляем DataGridView плавно прокрутиться к этому элементу
dgAddresses.ScrollIntoView(lastItem);
}
}

});
}
}
catch (IOException) { }
catch (Exception ex)
{

```

```

System.Diagnostics.Debug.WriteLine($"[Log Read Error] {ex.Message}");
}
}

protected override void OnClosing(System.ComponentModel.CancelEventArgs e)
{
    // Запоминаем выбор пользователя перед выходом
    Properties.Settings.Default.AutoMonitoringEnabled = chkEnableMonitoring.IsChecked ?? false;
    Properties.Settings.Default.Save(); // Записываем на диск

    // Твой старый код очистки таймеров
    if (_gameCheckTimer != null)
    {
        _gameCheckTimer.Stop();
        _gameCheckTimer.Dispose();
    }
    _logReadTimer?.Dispose();

    base.OnClosing(e);
}

// Хранилище для WAN-исключений (чтобы метод события их видел)
private List<IPclass> _routerWanExclusions = new List<IPclass>();
private List<MikrotikConfig> _activeRouters = new List<MikrotikConfig>();

private async Task SetupAndSyncMikrotiksAsync()
{
    tbStatusProp.Text = "Синхронизация роутеров...";
    _activeRouters = RouterStorage.Load();
    if (_activeRouters.Count == 0) return;

    _routerWanExclusions.Clear();

    // Глобальная база данных: [IP-адрес -> Самый свежий оставшийся таймаут среди всех роутеров]
    var globalTimeoutMap = new Dictionary<string, string>();

    // ЭТАП 1: Собираем данные и точные таймауты со всех роутеров
    foreach (var config in _activeRouters)
    {
        var client = new MikrotikRestClient(config);
        if (await client.TestConnectionAsync() != ConnectionStatus.Success) continue;

        // Забираем WAN IP для исключений
        var wanIp = await client.GetInterfaceIpAsync();
        if (wanIp != null)
        {
            wanIp.PoolSize = 32;
            _routerWanExclusions.Add(wanIp);
        }

        // Скачиваем словарь [IP -> Таймаут] с конкретного роутера
        var routerAddresses = await client.GetActiveAddressesWithTimeoutsAsync(config.TargetAddressList);
        foreach (var kvp in routerAddresses)
        {
            string ip = kvp.Key;
            string currentTimeout = kvp.Value;

            // Если адрес статический (таймаут пустой), он имеет приоритет — оставляем пустым.
            // Если адрес динамический, сохраняем его таймаут в общий котел.
            if (!globalTimeoutMap.ContainsKey(ip) || string.IsNullOrEmpty(currentTimeout))
            {
                globalTimeoutMap[ip] = currentTimeout;
            }
        }
    }

    // ЭТАП 2: Наполняем твой главный EDList на форме для фильтрации логов
    foreach (var ipStr in globalTimeoutMap.Keys)

```

```

{
IPclass? ipObj = IPclass.Parse(ipStr);
if (ipObj != null)
{
EDList.AddAddress(ipObj); // Твой умный список впитывает базовые адреса
}
}

// ЭТАП 3: Рассылаем недостающие адреса на роутеры С КОРРЕКТНЫМ ВРЕМЕНЕМ ЖИЗНИ
foreach (var config in _activeRouters)
{
var client = new MikrotikRestClient(config);
if (await client.TestConnectionAsync() != ConnectionStatus.Success) continue;

// Повторно запрашиваем точечную базу этого роутера, чтобы понять, чего ему не хватает
var currentRouterMap = await client.GetActiveAddressesWithTimeoutsAsync(config.TargetAddressList);

foreach (var masterKvp in globalTimeoutMap)
{
string masterIpStr = masterKvp.Key;
string preciseTimeout = masterKvp.Value; // Точное оставшееся время (например "4d12:00:15")

// Если на этом роутере этой записи вообще нет – пушим её с точным скопированным временем жизни!
if (!currentRouterMap.ContainsKey(masterIpStr))
{
// Если таймаут пустой (статическая запись), шлем null.
// Если динамический – пробрасываем точную строку времени из Mikrotik!
string? finalTimeoutParam = string.IsNullOrEmpty(preciseTimeout) ? null : preciseTimeout;

await client.AddAddressAsync(masterIpStr, config.TargetAddressList, finalTimeoutParam);
}
}

// ЭТАП 4: Привязываем событие живого отслеживания
EDList.OnAddressAdded += EDList_OnAddressAdded;

// Обновляем DataGridView на экране
Dispatcher.Invoke(() =>
{
dgAddresses.ItemsSource = null;
dgAddresses.ItemsSource = EDList.ipTable;
txtTotalAddressesCount.Text = EDList.ipTable.Count.ToString();
});

tbStatusProp.Text = "Роутеры синхронизированы. Мониторинг готов.";
}

// Убрали 'async', теперь это обычный быстрый метод, который не ругает компилятор
private void EDList_OnAddressAdded(IPclass newAddress)
{
foreach (var wan in _routerWanExclusions)
{
if (wan.IPinPool(newAddress)) return;
}

string ipStr = newAddress.ToString();
string? timeoutParam = (newAddress.PoolSize == 32 || newAddress.PoolSize == 0) ? "7d" : null;

// ОБНОВЛЕНИЕ UI: Выполняем в потоке интерфейса
Dispatcher.Invoke(() =>
{
// 1. Динамически обновляем счетчик количества адресов в твоём wiseIPList таблицы
// Предполагаем, что у тебя коллекция в EDList называется ipTable или аналогично
txtTotalAddressesCount.Text = EDList.ipTable.Count.ToString();

// 2. Добавляем запись в ленту "живого лога" на правой панели с отметкой времени
string timeStamp = DateTime.Now.ToString("HH:mm:ss");

```

```

string logMessage = $"[{timeStamp}] Обнаружен хост: {ipStr} -> отправка на Mikrotik";

lstLiveActivity.Items.Insert(0, logMessage); // Добавляем наверх, чтобы свежие события были видны сразу

// Ограничим размер истории в UI (например, хранить последние 50 записей, чтобы не забивать память)
if (lstLiveActivity.Items.Count > 50)
{
    lstLiveActivity.Items.RemoveAt(lstLiveActivity.Items.Count - 1);
}
});

// Сетевая фоновая отправка на роутеры
Task.Run(async () =>
{
    foreach (var config in _activeRouters)
    {
        try
        {
            var client = new MikrotikRestClient(config);
            await client.AddAddressAsync(ipStr, config.TargetAddressList, timeoutParam).ConfigureAwait(false);
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine($"[Rest API] Ошибка отправки на {config.Name}: {ex.Message}");
        }
    }
});

private void CreatePVK()
{
    string filename_pvt = WorkingDir + "\\keys\\private.key";
    string filename_pub = WorkingDir + "\\keys\\public.key";
    if (!File.Exists(filename_pvt))
    {
        if (File.Exists(filename_pub)) File.Delete(filename_pub);
        var keygen = new SshKeyGenerator.SshKeyGenerator(2048); // 2048 — длина ключа в битах

        var privateKey = keygen.ToPrivateKey();
        // Console.WriteLine(privateKey);

        var publicSshKey = keygen.ToRfcPublicKey();
        // Console.WriteLine(publicSshKey);
        FileStream fs = File.Create(filename_pvt);
        byte[] buf = System.Text.Encoding.UTF8.GetBytes(privateKey);
        fs.Write(buf, 0, buf.Length);
        fs.Flush();
        fs.Close();
        //
        fs = File.Create(filename_pub);
        buf = System.Text.Encoding.UTF8.GetBytes(publicSshKey);
        fs.Write(buf, 0, buf.Length);
        fs.Flush();
        fs.Close();
    }
}

// 1. СОБЫТИЕ: СТАРТ ПАРСИНГА (Узнаем общее количество файлов)
private void EDList_OnParseStart(object sender, WIP_Parse_StartEventArgs e)
{
    Dispatcher.Invoke(() =>
    {
        // Очищаем окно живого лога перед началом нового пакетного анализа
        lstLiveActivity.Items.Clear();

        string message = $"[СИСТЕМА] Запущен пакетный анализ. Всего файлов для обработки: {e.FilesCount}";
    });
}

```

```

lstLiveActivity.Items.Insert(0, message);

tbStatusProp.Text = "Пакетный анализ...";
tbStatusVal.Text = $"Обработано файлов: 0 из {e.FilesCount}";
});
}

// 2. СОБЫТИЕ: ПРОГРЕСС ПАРСИНГА (Срабатывает при переходе к каждому следующему файлу)
// Внутри твоего метода EDList_OnParseProceed (Прогресс парсинга)
private void EDList_OnParseProceed(object sender, WIP_Parse_ProceedEventArgs e)
{
    Dispatcher.BeginInvoke(new Action(() =>
    {
        string shortName = System.IO.Path.GetFileName(e.FileName);

        // В RELEASE-МОДЕ: Если файлов ОЧЕНЬ много, частая запись в ListBox
        // вызывает коллизии памяти. Обновляем ТОЛЬКО строку статуса внизу!
        tbStatusVal.Text = $"Анализ: {shortName} [{e.FileIndex}]";

        // Добавление в ListBox для логов делаем только для вех,
        // например для каждого 5-го файла, чтобы UI не падал в гонку потоков
        if (e.FileIndex % 5 == 0)
        {
            lstLiveActivity.Items.Insert(0, $"[{DateTime.Now:HH:mm:ss}] Обработка пакета [{e.FileIndex}]");
        }
    }));
}

// 3. СОБЫТИЕ: ЗАВЕРШЕНИЕ ПАРСИНГА
private void EDList_OnParseComplete(object sender, EventArgs e)
{
    Dispatcher.Invoke(() =>
    {
        string endMessage = $"[СИСТЕМА] Пакетный анализ полностью завершен. Сводная таблица обновлена.";
        lstLiveActivity.Items.Insert(0, endMessage);

        tbStatusProp.Text = "Анализ завершен успешно.";
        // Выводим итоговое количество элементов в твоём wiseIPList таблицы
        tbStatusVal.Text = $"Итого уникальных сетей в базе: {EDList.ipTable.Count}";
    });
}

private void MenuItem_Click(object sender, RoutedEventArgs e)
{
}

private void LogAnalyze(object sender, RoutedEventArgs e)
{
    EDList.ParseEDLogsAsync(EDLogs);
}

private void SaveList(object sender, RoutedEventArgs e)
{
    SaveList(false);
}

private void SaveListMikrotik(object sender, RoutedEventArgs e)
{
    SaveList(true);
}

private void SaveList(bool forMikrotik)
{
    SaveFileDialog SFD = new SaveFileDialog();
    SFD.InitialDirectory = WorkingDir;
    SFD.AddExtension = true;
    SFD.CreatePrompt = true;
    if (forMikrotik)
    {
        SFD.Title = "Сохранить список адресов для Mikrotik";
        SFD.Filter = "Mikrotik resource(*.RSC)|*.rsc|Текстовый файл(*.txt)|*.txt";
    }
}

```

```

else
{
SFD.Title = "Сохранить список адресов";
SFD.Filter = "Текстовый файл(*.txt)|*.txt";
}
if (SFD.ShowDialog() == true)
{
EDList.SaveTo(SFD.FileName, forMikrotik);
}
}

private void MenuItem_Click_1(object sender, RoutedEventArgs e)
{
var a = new frmMikrotikRestConfig();
a.ShowDialog();
}

private void ChkEnableMonitoring_Checked(object sender, RoutedEventArgs e)
{
// Защита от падения во время инициализации окна
if (chkEnableMonitoring == null || tbStatusProp == null) return;

if (_isEliteRunning)
{
StartLiveLogMonitoring();
}
tbStatusProp.Text = "Автоматическое отслеживание логов включено.";
}

private void ChkEnableMonitoring_Unchecked(object sender, RoutedEventArgs e)
{
if (chkEnableMonitoring == null || tbStatusProp == null) return;

StopLiveLogMonitoring();
tbStatusProp.Text = "Автоматическое отслеживание логов отключено.";
}

private void BtnOpenSettings_Click(object sender, RoutedEventArgs e)
{
// Открываем созданную ранее отдельную форму настроек REST API
var settingsWindow = new frmMikrotikRestConfig { Owner = this };
settingsWindow.ShowDialog();
}

private async void BtnOldParse_Click(object sender, RoutedEventArgs e)
{
// Блокируем UI от повторных нажатий на время обработки истории
btnOldParse.IsEnabled = false;
chkEnableMonitoring.IsEnabled = false;

// 1. Отвязываем событие живой отправки хостов на время чтения истории,
// чтобы не флудить в сеть одиночными запросами в процессе парсинга
EDList.OnAddressAdded -= EDList_OnAddressAdded;

// 2. Запускаем твой тяжелый метод анализа папки в фоновом потоке ОС.
// Интерфейс приложения при этом остается полностью живым и отзывчивым!
await Task.Run(() =>
{
// Вызываем твой ОРИГИНАЛЬНЫЙ метод пакетного анализа папки с логами
// Пример: EDList.ParseAllHistoryFiles();
EDList.ParseEDLogs(Settings.Default.EDLogFolder, false);
});

// 3. Анализ завершен. Один раз красиво обновляем DataGrid и счетчик на экране
dgAddresses.ItemsSource = null;
dgAddresses.ItemsSource = EDList.ipTable;

```

```

txtTotalAddressesCount.Text = EDList.ipTable.Count.ToString();

// 4. ПРЯМОЙ ПУШ: Отправляем готовый список на роутеры без лишних проверок
tbStatusProp.Text = "Синхронизация списков на роутерах...";
var routers = RouterStorage.Load();
var syncCore = new EDIPSearch.Core.MikrotikSyncCore();

// Сетевую отправку тоже делаем в фоне, чтобы форма не моргала
await Task.Run(async () =>
{
    await syncCore.PushNewAddressesToRoutersAsync(routers, EDList).ConfigureAwait(false);
});

// 5. Возвращаем событие живого мониторинга логов на место для отлова IP во время игры
EDList.OnAddressAdded += EDList_OnAddressAdded;

// Разблокируем интерфейс
btnOldParse.IsEnabled = true;
chkEnableMonitoring.IsEnabled = true;
tbStatusProp.Text = "Готово.";
tbStatusVal.Text = $"В базе роутеров обновлено элементов: {EDList.ipTable.Count}";
}

private void BtnOldSettings_Click(object sender, RoutedEventArgs e)
{
    var a = new frmSettings();
    if(a.ShowDialog()==true)
    {
        Settings.Default.Save();
    }
}

// ... внутри класса MainWindow ...

/// <summary>
/// Продвинутый автоматический поиск папки журналов Elite Dangerous
/// </summary>
/// <summary>
/// Безупречное автоопределение папки сетевых журналов netLog в каталоге установки игры
/// </summary>
private string AutoDetectEliteDangerousPath()
{
    // --- ШАГ 1: Поиск в системном реестре установленных программ Windows (Работает при закрытой игре)
    ---
    string[] registryPaths = new string[]
    {
        // 1. Путь для 64-битных систем (Steam / Epic / Frontier Launcher)
        @"SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\Elite Dangerous",
        @"SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\Elite Dangerous",
        // 2. Альтернативный ключ Epic Games Store
        @"SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\Eden", // Внутреннее имя Элиты в Epic Store
        // 3. Прямые пути регистрации команд запуска Windows (App Paths)
        @"SOFTWARE\Microsoft\Windows\CurrentVersion\App Paths\EliteDangerous64.exe"
    };

    foreach (var regPath in registryPaths)
    {
        try
        {
            // Ищем сначала в ветке LocalMachine, затем в CurrentUser
            using (var key = Registry.LocalMachine.OpenSubKey(regPath) ??
                Registry.CurrentUser.OpenSubKey(regPath))
            {
                if (key != null)
                {
                    // Для Uninstall-ключей забираем InstallLocation (папку установки)
                    string? installDir = key.GetValue("InstallLocation") as string;

```

```

// Если это был ключ App Paths, забираем значение по умолчанию (путь к .exe)
if (string.IsNullOrEmpty(installDir))
{
    string? exePath = key.GetValue("") as string; // Значение (По умолчанию)
    if (!string.IsNullOrEmpty(exePath)) installDir = System.IO.Path.GetDirectoryName(exePath);
}

if (!string.IsNullOrEmpty(installDir))
{
    // Достаиваем сквозной путь до сетевой папки Logs Одиссеи
    string logPath = System.IO.Path.Combine(installDir, "Products", "elite-dangerous-odyssey-64",
    "Logs");
    if (Directory.Exists(logPath))
    {
        System.Diagnostics.Debug.WriteLine($"[Registry AutoDetect] Папка логов найдена в реестре Windows:
        {logPath}");
        return logPath;
    }
}
}
}
}
catch { }
}

// --- ШАГ 2: Резервный динамический перехват (Если реестр пуст, но игра запущена прямо сейчас) ---
try
{
    Process[] processes = Process.GetProcessesByName("EliteDangerous64");
    if (processes.Length > 0)
    {
        string? exeFullPath = processes[0].MainModule?.FileName;
        if (!string.IsNullOrEmpty(exeFullPath))
        {
            string? productDirectory = System.IO.Path.GetDirectoryName(exeFullPath);
            if (!string.IsNullOrEmpty(productDirectory))
            {
                string dynamicLogPath = System.IO.Path.Combine(productDirectory, "Logs");
                if (Directory.Exists(dynamicLogPath))
                {
                    System.Diagnostics.Debug.WriteLine($"[Process AutoDetect] Путь перехвачен от запущенной игры:
                    {dynamicLogPath}");
                    return dynamicLogPath;
                }
            }
        }
    }
}
catch { }

return string.Empty; // Если ничего не помогло, пилот выберет папку сам через проводник □
}

```

```

---wiseIPList.cs---
using System;
using System.Collections.Generic;
using System.ComponentModel.Design;
using System.IO;
using System.Linq;
using System.Net;
using System.Net.Security;
using System.Runtime.CompilerServices;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using Windows.ApplicationModel.Contacts;
using Windows.UI.Input.Inking.Preview;
using static ipinpool.wiseIPList;

```

```

namespace ipinpool
{
public class wiseIPList
{

public List<IPclass> ipTable { get; set; } = new List<IPclass>();
public List<IPclass> PoolTable { get; set; } = new List<IPclass>();
public List<IPclass> Filter { get; set; } = new List<IPclass>(); //Адреса исключаемые при разборе
лога
public void ParseFile(string filename)
{
if (!File.Exists(filename)) return;
//FileStream rdr = TextReader(filename);
FileStream frdr = File.Open(filename, FileMode.Open, FileAccess.Read, FileShare.ReadWrite);
TextReader reader = new StreamReader(frdr);
string? line = reader.ReadLine();

while (line != null)
{
if (IPclass.TryParse(line, out IPclass ip))
{
if (ip.IsPool) { PoolTable.Add(ip); }
else { ipTable.Add(ip); }

}
line = reader.ReadLine();
}
reader.Close();
frdr.Close();
}
public void Sort(ref List<IPclass> list)
{
for (int i = 0; i < list.Count-1; i++)
for(int j=i+1; j<list.Count; j++)
{
if (list[i].IsAbove(list[j]))
{
IPclass ip = list[i];
list[i] = list[j];
list[j] = ip;
}
}
}
public void AddPool(IPclass? ip)
{
if(ip!=null)
{
if(ip.IsPool)
{
PoolTable.Add(ip);
}
}
}

public void AddAddress(IPclass? ip)
{
List<IPclass> iptable = this.ipTable;
AddAddressTo(ip, ref iptable);
}
public void AddAddressTo(IPclass? ip, ref List<IPclass> iptable)
{
if(ip==null) return;
// if (ipTable.IndexOf(ip) > 0) return;
// if(Filter.IndexOf(ip) > 0) return;
if (AddrInTable(ip, iptable)) return;
if (AddrInTable(ip, Filter)) return;
iptable.Add(ip);
// 3.Вызываем событие, если на него кто-то подписался
OnAddressAdded?.Invoke(ip);
}
private bool AddrInTable(IPclass ip, List<IPclass> Table)
{

```

```

foreach(IPclass p in Table)
{
bool r = p.IPinPool(ip);
if (r) return true;
}
return false;
}
public void ParseEDLogs(string dir, bool newlist = true)
{
//F:\SteamLibrary\steamapps\common\Elite Dangerous\Products\elite-dangerous-odyssey-64\Logs
string local_dir = "F:\\SteamLibrary\\steamapps\\common\\Elite Dangerous\\Products\\elite-
dangerous-odyssey-64\\Logs";
if(dir!=string.Empty) local_dir = dir.ToLower();
int index = 0;
if(Directory.Exists(local_dir))
{
if(newlist) ipTable.Clear();
string[] files = Directory.GetFiles(local_dir, "*.log");
OnParseStart?.Invoke(this, new WIP_Parse_StartEventArgs(files.Length));
foreach (string file in files)
{

OnParseProceed?.Invoke(this, new WIP_Parse_ProceedEventArgs(index, file));
parseFile(file);
index++;
}
}
OnParseComplete?.Invoke(this, new EventArgs());
}
public List<IPclass> ParseString(string text)
{
List<IPclass> rez = new List<IPclass>();
string[] strings = text.Split("\n");
foreach (string s in strings)
{
string ip_str = ipSearch(s);
if (ip_str != null)
{
AddAddressTo(IPclass.Parse(ip_str), ref rez);
}
}
return rez;
}
public void ParseEDLogsAsync(string dir, bool newlist = true)
{
//await ParseEDLogs(dir, newlist);
Task task = new Task(()=>ParseEDLogs(dir, newlist));
task.Start();
}

private void parseFile(string filename)
{
try
{
FileStream fs = File.Open(filename, FileMode.Open, FileAccess.Read, FileShare.ReadWrite);
TextReader reader = new StreamReader(fs);
string? currentLine = String.Empty;
currentLine = reader.ReadLine();

while (currentLine != null)
{
// 1. АВТОМАТИЧЕСКИЙ ДИНАМИЧЕСКИЙ ФИЛЬТР (Убираем рутину пользователя)
// Ищем маркеры локального внешнего адреса или STUN-точки выхода
if (currentLine.Contains("WAN:") ||
currentLine.Contains("STUN mapped address is"))

{
// Выдергиваем IP-адрес из этой конкретной технической строки
string filterIpStr = ipSearch(currentLine);
if (!string.IsNullOrEmpty(filterIpStr))
{
IPclass? filterIpObj = IPclass.Parse($"{filterIpStr}/32");
if (filterIpObj != null)
{

```

```

// Если этого адреса еще нет в черном списке — добавляем его в Filter!
// Метод AddrInTable у тебя уже написан на странице 2.
if (!AddrInTable(filterIpObj, this.Filter))
{
    this.Filter.Add(filterIpObj);
    // Можно отправить лог в отладку, чтобы видеть, что автофильтр сработал
    System.Diagnostics.Debug.WriteLine($"[AutoFilter] Локальный/STUN адрес изолирован: {filterIpStr}");
}
}
}

// 2. БОЕВОЙ ПАРСИНГ ИГРОВЫХ ХОСТОВ (Остается штатным)
string ip_str = ipSearch(currentLine);
if (!string.IsNullOrEmpty(ip_str))
{
    // Твой метод AddAddress сам сравнит адрес с обновленным списком Filter
    // и заблокирует добавление твоего WAN/STUN IP в общую таблицу!
    AddAddress(IPclass.Parse(ip_str));
}

currentLine = reader.ReadLine();
}
reader.Close();
fs.Close();
}
catch (Exception e)
{
    Console.WriteLine(e.ToString());
}
}

private string ipSearch(string logEntry)
{
    if (string.IsNullOrEmpty(logEntry)) return String.Empty;
    //Logwrite(logEntry);
    // ИСПРАВЛЕНО: \b требует, чтобы перед первой и после последней цифры шла граница слова.
    // {1,3} означает строго от 1 до 3 цифр в каждом октете.
    // Теперь число 4.0.0.1103 будет полностью ПРОИГНОРИРОВАНО, так как 1103 содержит 4 цифры!
    Regex reg = new Regex(@"\b(?:[0-9]{1,3}\.){3}[0-9]{1,3}\b");

    if (reg.IsMatch(logEntry))
    {
        string rez = reg.Match(logEntry).ToString();
        // Logwrite("[!IPF!] =>" + rez);
        return rez;
    }
    return String.Empty;
}

private static void Logwrite(string outstr)
{
    var fl = File.AppendText("f:\\!log.txt");
    fl.WriteLine(outstr);
    fl.Flush();
    fl.Close();
}

public void SaveTo(string filename, bool mikrotik_prep = true)
{
    try
    {
        StreamWriter wr = new StreamWriter(filename);

        foreach (IPclass ip in ipTable)
        {
            string ws = string.Empty;
            if(mikrotik_prep)
            {
                ws = "/ip firewall address-list add address=" + ip.ToString() + " list=ED";
            }
            else
            {
                ws = ip.ToString();
            }
            wr.WriteLine(ws);
        }
    }
}

```

```

    }
    wr.Flush();
    wr.Close();
    }
    catch (Exception e) { Console.WriteLine(e.Message); }

}

public bool LoadFilters(string filename)
{
    //список фильтров представляет собой текстовый файл вида IP/Mask
    if (File.Exists(filename))
    {
        try
        {
            this.Filter.Clear();
            TextReader reader = new StreamReader(filename);
            string? line = reader.ReadLine();
            while (line != null)
            {
                if(line.IndexOf("#")>-1)
                {
                    line = line.Substring(0, line.IndexOf("#"));
                }
                IPclass? ip = IPclass.Parse(line);
                if (ip != null)
                {
                    if(!AddrInTable(ip, this.Filter)) this.Filter.Add(ip);
                }
                line = reader.ReadLine();
            }
            reader.Close();
            return true;
        }
        catch
        { return false; }

    }
    else return false;
}

public void SetDefaultFilters(string filename)
{
    try
    {
        TextWriter writer = new StreamWriter(filename);
        writer.WriteLine("#cut off 0.0.0.0/32 ");
        writer.WriteLine("0.0.0.0/32");
        writer.WriteLine("#cut off local network areas");
        writer.WriteLine("10.0.0.0/8");
        writer.WriteLine("172.16.0.0/12");
        writer.WriteLine("192.168.0.0/16");
        writer.WriteLine("#cut off routers");

        writer.WriteLine("#place adresses to exclude below IP/Mask");
        writer.Flush();
        writer.Close();
    }
    catch { }
}

public delegate void LogParseStart(object sender, WIP_Parse_StartEventArgs e);
public delegate void LogParseProceed(object sender, WIP_Parse_ProceedEventArgs e);
public delegate void LogParseComplete(object sender, EventArgs e);
public delegate void AddressAddedEventHandler(IPclass newAddress);

public event LogParseStart? OnParseStart;
public event LogParseProceed? OnParseProceed;
public event LogParseComplete? OnParseComplete;
public event AddressAddedEventHandler? OnAddressAdded;
}

public class WIP_Parse_StartEventArgs:EventArgs
{
    public int FilesCount { get; private set; } = 0;
}

```

```

public WIP_Parse_StartEventArgs(int filesCount)
{
    this.FilesCount = filesCount;
}
}
public class WIP_Parse_ProceedEventArgs : EventArgs
{
    public int FileIndex { get; private set; } = 0;
    public string Filename { get; private set; } = string.Empty;
    public WIP_Parse_ProceedEventArgs( int fileIndex, string filename)
    {
        FileIndex = fileIndex;
        Filename = filename;
    }
}

public class IPclass
{
    // Заменяем автосвойство на жесткое приватное поле.
    // Это на 100% заставит компилятор в Release выделять новый изолированный массив для каждого
    // объекта!
    internal byte[] _ipa = new byte[4];

    // Публичное свойство теперь просто ссылается на наше изолированное поле
    public byte[] IPA
    {
        get { return _ipa; }
        set { _ipa = value; }
    }

    // Чистый, стандартный конструктор
    public IPclass()
    {
        // Принудительно выделяем персональный массив из 4 байт при рождении каждого объекта
        _ipa = new byte[4] { 0, 0, 0, 0 };
    }

    public int Port { get; set; } = 0;
    public int PoolSize { get; set; } = 32;
    public bool IsPool { get { return PoolSize < 32; } }

    private int _inPoolCounter = 0;
    public int InPoolCounter { get { return _inPoolCounter; } }
    // Измени заголовок и концовку метода TryParse на обычный возврат объекта
    private static void Logwrite(IPclass pc)
    {
        Logwrite(pc._ipa[0].ToString()+"."+ pc._ipa[1].ToString() + "." + pc._ipa[2].ToString() + "." +
        pc._ipa[3].ToString() );
        Logwrite("---//---");
    }
    private static void Logwrite(string out_str)
    {
        var fl = File.AppendText("f:\\!log.txt");

        fl.WriteLine(out_str);
        fl.Flush();
        fl.Close();
    }

    [MethodImpl(MethodImplOptions.NoInlining)]
    public static IPclass? Parse(string str)
    {
        if (string.IsNullOrEmpty(str)) return null;
        if (str.Trim().ToLower().IndexOf("#") > -1) return null;
        // Logwrite(str);
        string tmps = str.ToLower().Replace("add address=", "");
        int ioflist = tmps.IndexOf("list");
        if (ioflist > -1) tmps = tmps.Substring(0, ioflist);
        tmps = tmps.Trim();
        string ipstr = tmps;

```

```

int poolSize = 32;
int port = 0;

if (tmps.IndexOf("/") > 0)
{
var parts = tmps.Split('/');
ipstr = parts[0].Trim();
if (parts.Length > 1 && int.TryParse(parts[1], out int ps))
poolSize = ps;
}

if (tmps.IndexOf(":") > 0)
{
var parts = tmps.Split(':');
ipstr = parts[0].Trim();
if (parts.Length > 1 && int.TryParse(parts[1], out int p))
port = p;
}

string[] octets = ipstr.Split('.');
if (octets.Length != 4) return null;

if (!byte.TryParse(octets[0], out byte b0) ||
!byte.TryParse(octets[1], out byte b1) ||
!byte.TryParse(octets[2], out byte b2) ||
!byte.TryParse(octets[3], out byte b3))
return null;

// Создаём объект и сразу заполняем его внутренний массив
IPclass pc = new IPclass();
pc.PoolSize = poolSize;
pc.Port = port;
pc._ipa[0] = b0;
pc._ipa[1] = b1;
pc._ipa[2] = b2;
pc._ipa[3] = b3;
// Logwrite(pc);
return pc;
}

// Старый метод TryParse можно просто перенаправить на новый Parse для совместимости:
public static bool TryParse(string str, out IPclass pc)
{
var res = Parse(str);
pc = res ?? new IPclass();
return res != null;
}

public override string ToString()
{
string rez = string.Empty;
foreach (byte b in IPA)
{
if (rez.Length > 0) { rez += "." + b.ToString(); } else { rez = b.ToString(); }
}

// ИСПРАВЛЕНО: Выводим маску / только если это реальный пул (подсеть)!
// Для обычных хостов /32 слэш больше выводиться не будет, что спасет JIT от сбоя.
if (PoolSize < 32 && PoolSize > 0)
{
rez += "/" + PoolSize.ToString();
}

if (Port > 0)
{
rez += ":" + Port.ToString();
}
return rez;
}

```

```

public bool LongIPMask { get; set; } = false;
public bool IPinPool(IPclass ip)
{
    if (this.PoolSize < 8)
    {
        byte hr = getHighRange(this.IPA[0], this.PoolSize);
        if ((ip.IPA[0] >= this.IPA[0]) && (ip.IPA[0] <= hr))
        {
            _inPoolCounter++;
            return true;
        }
        return false;
    }
    if ((this.PoolSize >= 8) && (this.PoolSize < 16))
    {
        byte hr = getHighRange(this.IPA[1], this.PoolSize-8);
        if (ip.IPA[0] == this.IPA[0])
        if ((ip.IPA[1] >= this.IPA[1]) && (ip.IPA[1] <= hr))
        {
            _inPoolCounter++;
            return true;
        }
        return false;
    }
    if ((this.PoolSize >= 16) && (this.PoolSize < 24))
    {
        byte hr = getHighRange(this.IPA[2], this.PoolSize - 16);
        if ((ip.IPA[0] == this.IPA[0]) && (ip.IPA[1] == this.IPA[1]))
        if ((ip.IPA[2] >= this.IPA[2]) && (ip.IPA[2] <= hr))
        {
            _inPoolCounter++;
            return true;
        }
        return false;
    }
    if ((this.PoolSize >= 24) && (this.PoolSize < 32))
    {
        byte hr = getHighRange(this.IPA[3], this.PoolSize - 24);
        if ((ip.IPA[0] == this.IPA[0]) && (ip.IPA[1] == this.IPA[1]) && (ip.IPA[2] == this.IPA[2]))
        if ((ip.IPA[3] >= this.IPA[3]) && (ip.IPA[3] <= hr))
        {
            _inPoolCounter++;
            return true;
        }
        return false;
    }
    if (this.PoolSize >= 32)
    {
        for (int i = 0; i < this.IPA.Length; i++)
        {
            if (this.IPA[i] != ip.IPA[i]) return false;
        }
        _inPoolCounter++;
        return true;
    }

    return false;
}

private byte getHighRange(byte low, int mask)
{
    // Защита от переполнения: если из-за инлайнинга JIT сюда прилетит
    // некорректный индекс маски, мы жестко ограничиваем его границами массива maskTable
    int safeMask = mask;
    if (safeMask < 0) safeMask = 0;
    if (safeMask > 7) safeMask = 7;

    byte rez = 0;
    byte[] maskTable = { 255, 127, 63, 31, 15, 7, 3, 1 };

    rez = (byte)(low + maskTable[safeMask]);
    return rez;
}

```

```

public bool IsAbove(IPclass ip)
{
for (int i = 0; i < this.IPA.Length; i++)
{
if(ip.IPA[i] < this.IPA[i])
{
return true;
}
if(ip.IPA[i] > this.IPA[i])
{
return false;
}
}
return false;
}
public string IP { get {
string rez = string.Empty;
foreach (byte b in this.IPA)
{
if(rez==string.Empty)
{
rez = b.ToString();
}
else
{
rez += "." + b.ToString();
}
}
return rez;
} }
public string Mask { get {
if (!LongIPMask)
{ return this.PoolSize.ToString(); }
else return IPMask.GetStrIPMask(this.PoolSize);
} }
public string Size
{
get
{
if (PoolSize == 32) return "1";
if (PoolSize == 31) return "2";
}
}

// Для всех остальных подсетей (30, 29, 24 и т.д.) вычитаем 2 служебных адреса
long totalAddresses = (long)Math.Pow(2, 32 - PoolSize);
long usableAddresses = totalAddresses - 2;

return usableAddresses.ToString();
}
}

}

public class IPMask
{
static byte[] masks = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32 };

static public string GetStrIPMask(int mask)
{
string rez = string.Empty;
if (mask < 0 || mask > 31) return "255.255.255.255";
if(mask < 32&&mask >=24)
{
rez = "255.255.255.";
int m = mask-24;
rez += ByteMaskToValue(m);
}
if(mask<24&&mask>=16)
{
rez = "255.255.";
int m = mask - 16;
rez += ByteMaskToValue(m);
rez += ".0";
}
}
}

```

```
if(mask<16&&mask>=8)
{
    rez = "255.";
    int m = mask - 8;
    rez += ByteMaskToValue(m);
    rez += ".0.0";
}
return rez;
}
static string ByteMaskToValue(int mask)
{
    int b = 0;
    b = 2 ^ mask;
    return b.ToString();
}
}
```